



BITS Pilani
Pilani Campus

AIMLCZG537/DSECLZG537 Information Retrieval

Dr. Vijayalakshmi anand
BITS Pilani

Lecture Outline



Chapter 3: Dictionaries and Tolerant Retrieval

- Dictionary Data Structures
- Tolerant Retrieval

Slides are adapted from **Introduction to Information Retrieval; Manning, Raghavan**
<http://nlp.stanford.edu/IR-book>

Positional postings and Phrase queries



Phrase queries

- Want to be able to answer queries such as “**Stanford University**” – as a phrase
- The sentence “*I went to university at Stanford*” is not a match
- The sentence “*The inventor Stanford Ovshinsky never went to university*” is not a match
- For this, it no longer suffices to store only
 - `<term : docs>` entries

Handling Phrase queries



- Biword Indexes
- Positional Indexes

Biword Indexes

- Index every consecutive pair of terms in the text as a phrase
- For example, the text “Friends, Romans, Countrymen” would generate the biwords
 - *friends romans*
 - *romans countrymen*
- Each of these biwords is now a dictionary term
-

Biword indexes

- Longer phrases are processed by breaking them.
- ***stanford university palo alto*** can be broken into the Boolean query on biwords:
 - ***stanford university AND university palo AND palo alto***
-

Extended biwords

- Parse the indexed text and perform **part-of-speech-tagging** (POST).
- Bucket the terms into (say) Nouns (N) and articles/prepositions (X).
- Call any string of terms of the form NX^*N an extended biword.
 - Each such extended biword is now made a term in the dictionary.
- Example: cost overruns on a power plant

N N X X N
- Query processing: parse it into N's and X's
 - Segment query into extended biwords
 - “cost overruns” AND “overruns power” AND “power plant”
- Better results can be obtained by using more precise part-of-speech patterns that define which extended biwords

Issues for biword indexes

- False positives
- Index blowup due to bigger dictionary
- Biword indexes are not the standard solution (for all biwords) but can be part of a compound strategy.
- Blue house
-

Positional indexes

- In the postings, store, for each ***term*** the position(s) in which tokens of it appear:
 - ***<term***, number of docs containing ***term***;
 - *doc1*: frequency of the term; position1, position2 ... ;
 - *doc2*: frequency of the term; position1, position2 ... ;
 -

Positional indexes

to, 993427:

(1, 6: (7, 18, 33, 72, 86, 231);

2, 5: (1, 17, 74, 222, 255);

4, 5: (8, 16, 190, 429, 433);

5, 2: (363, 367);

7, 3: (13, 23, 191); . . .)

be, 178239:

(1, 2: (17, 25);

4, 5: (17, 191, 291, 430, 434);

5, 3: (14, 19, 101); . . .)

Positional indexes

- For phrase queries, we use a merge algorithm recursively at the document level
- This requires more than just equality
- Start with the least frequent term and then work, to further restrict the list of possible candidates
- Need to check that their positions of appearance in the document are compatible with the phrase query being evaluated
- This requires working out offsets between the words
- Positional Index can also be used for Proximity Searches.

Exercise - Positional Indexes



Exercise 2.9

[★]

Shown below is a portion of a positional index in the format: term: doc1: ⟨position1, position2, ...⟩; doc2: ⟨position1, position2, ...⟩; etc.

angels: 2: ⟨36,174,252,651⟩; 4: ⟨12,22,102,432⟩; 7: ⟨17⟩;
fools: 2: ⟨1,17,74,222⟩; 4: ⟨8,78,108,458⟩; 7: ⟨3,13,23,193⟩;
fear: 2: ⟨87,704,722,901⟩; 4: ⟨13,43,113,433⟩; 7: ⟨18,328,528⟩;
in: 2: ⟨3,37,76,444,851⟩; 4: ⟨10,20,110,470,500⟩; 7: ⟨5,15,25,195⟩;
rush: 2: ⟨2,66,194,321,702⟩; 4: ⟨9,69,149,429,569⟩; 7: ⟨4,14,404⟩;
to: 2: ⟨47,86,234,999⟩; 4: ⟨14,24,774,944⟩; 7: ⟨199,319,599,709⟩;
tread: 2: ⟨57,94,333⟩; 4: ⟨15,35,155⟩; 7: ⟨20,320⟩;
where: 2: ⟨67,124,393,1001⟩; 4: ⟨11,41,101,421,431⟩; 7: ⟨16,36,736⟩;

Which document(s) if any match each of the following queries, where each expression within quotes is a phrase query?

- a. "fools rush in"
- b. "fools rush in" AND "angels fear to tread"

Dictionaries and Tolerant Retrieval

Ch. 3

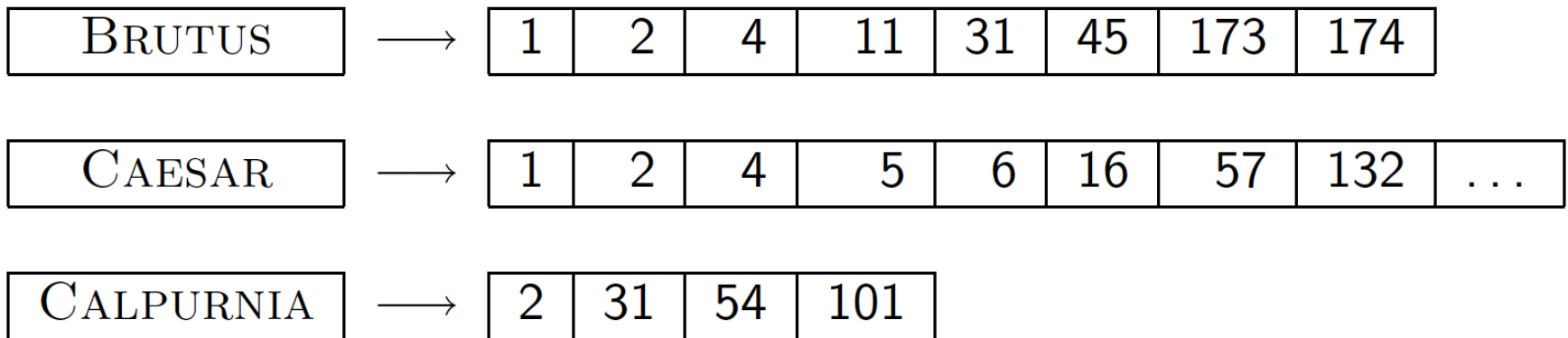
achieve

lead

- **Dictionary data structures**
 - data structures that help the search for terms in the vocabulary in an inverted index.
- **Tolerant retrieval**
 - techniques that are robust to typographical errors in the query, as well as alternative spellings.
 - Wild-card queries
 - Spelling correction
 -

Search structures for dictionaries

- The dictionary stores the term vocabulary, document frequency, pointers to each postings list ... **in what data structure?**



⋮

dictionary

postings

A simple dictionary

- An array of structures:

term	document frequency	pointer to postings list
a	656,265	→
aachen	65	→
...	...	...
zulu	221	→

char[20] int
 bytes 4/8 bytes 4/8 bytes

Postings

20

Dictionary data structures

- Two main choices:
 - Hashtables
 - Trees

The choice of solution (hashing, or search trees) depends on:

- No. of keys(Terms)
- Keys remain static or dynamic
- The relative frequencies with which various keys will be accessed.

Hashtables

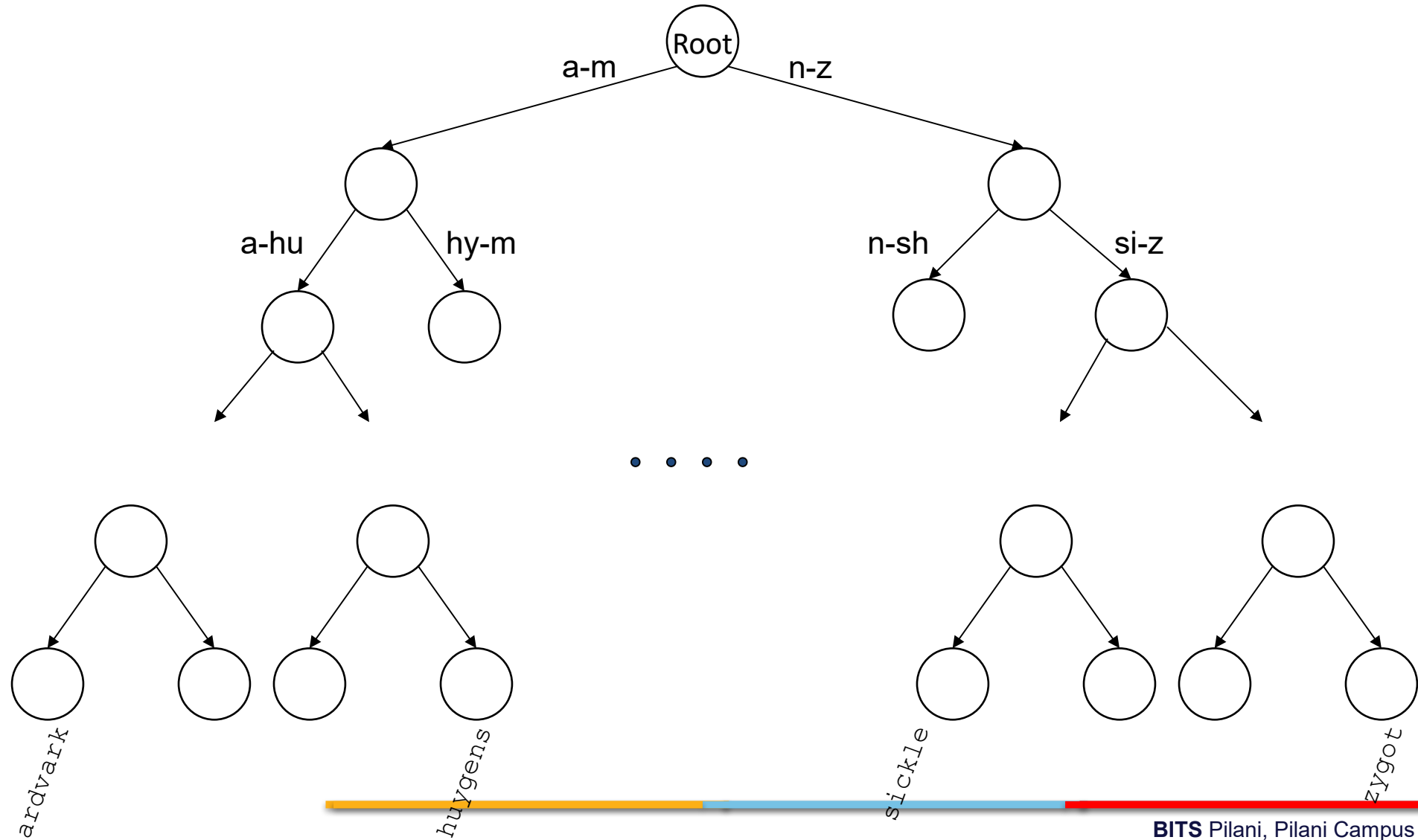
- Each vocabulary term is hashed to an integer
- **Pros:**
 - Lookup is faster than a tree: $O(1)$
- **Cons:**
 - No easy way to find minor variants:
 - judgment/judgement
 - No prefix search [tolerant retrieval]
 - If vocabulary keeps growing, need to occasionally do the expensive operation of rehashing

Trees



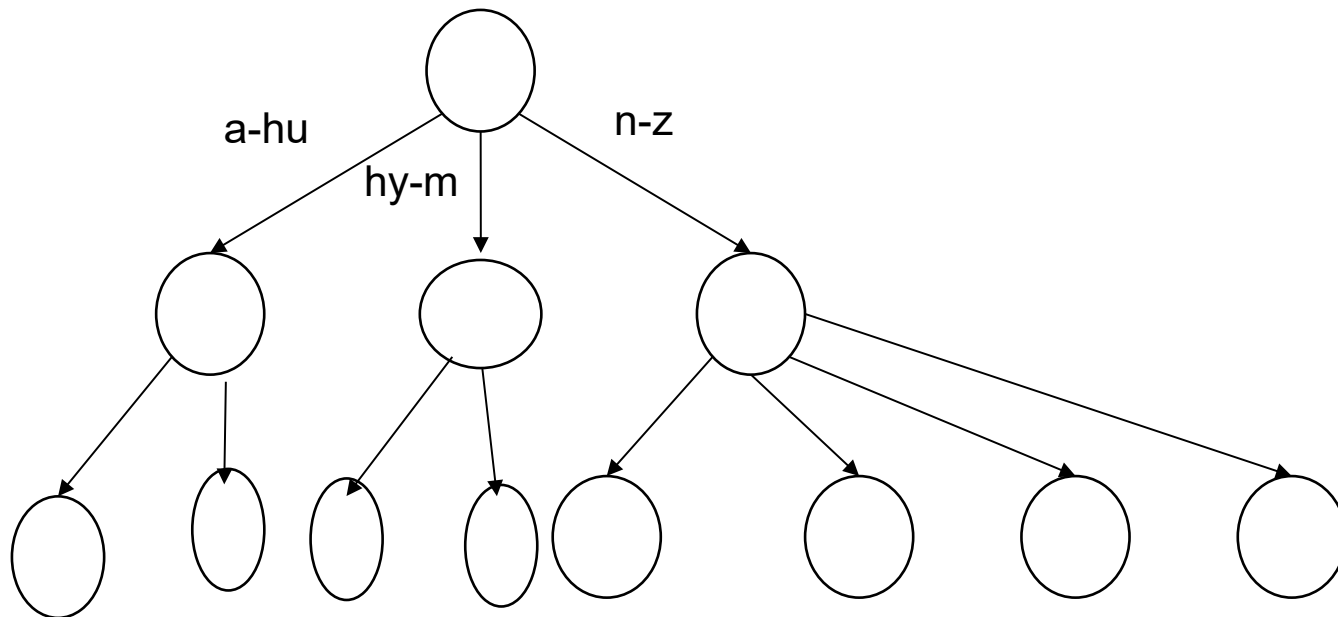
- Trees solve the prefix problem (find all terms starting with automat).
- Simplest tree: binary search tree
- Search is slightly slower than in hashes: $O(\log M)$, where M is the size of the vocabulary.
- $O(\log M)$ only holds for balanced trees

Tree: Binary Search Tree



Tree: B-tree

- Definition: Every internal node has a number of children in the interval $[a, b]$ where a, b are appropriate natural numbers, e.g., $[2, 4]$.



Trees

Pros:

- Solves the prefix problem (terms starting with *hyp*)

Cons:

- Slower: $O(\log M)$ [and this requires *balanced* tree]
- Rebalancing binary trees is expensive
 -

Tolerant Retrieval



- **Wild-card queries**
- Spelling correction
- Phonetic correction

Wild-card queries



- Uncertain of the spelling of a query term (e.g., Sydney vs. Sidney, which leads to the wildcard query S*dney)
- Seeking documents containing the variants (e.g., color vs. colour)
- Seeking documents containing variants of a term that would be caught by stemming, but is unsure whether the search engine performs stemming (e.g., judicial vs. judiciary, leading to the wildcard query judicia*)
- Uncertain of the correct rendition of a foreign word or phrase (e.g., the query Universit

Wild-card queries: *

- ***mon****: (trailing wildcard query) find all docs containing any word beginning with “mon”.
- Easy with binary tree (or B-tree) lexicon: retrieve all words in range: ***mon*** $\leq w <$ ***moo*** (*w is the set of terms which start with mon*)

Wild-card queries: *

- ***mon:** (leading wildcard query) find words ending in “mon”:
 - Maintain an additional B-tree (reverse B-tree)
 - Reverse B-tree is one in which each root-to-leaf path of the B-tree corresponds to a term in the dictionary written backwards:
 - Can retrieve all words in range: ***nom* ≤ *w* < *non***

Wild-card queries: *



- How can we handle *'s in the middle of query term?
 - **co*tion**
- We could look up **co*** AND ***tion** in a B-tree and the reverse B-tree and intersect the two term sets
 - Expensive
- The solution: transform wild-card queries so that the *'s occur at the end
- This gives rise to the **Permuterm** Index.

Wild-card queries: *



How to process the query ***se*ate AND fil*er*** ?



BITS Pilani
Pilani Campus

AIMLCZG537/DSECLZG537

Information Retrieval

Dr. Vijayalakshmi anand
BITS Pilani

Agenda



Wildcard Query processing

- ***Permuterm indexes***
- ***Kgram indexes***
- ***Tolerant reterival***



Wildcard Query Processing

- ***Permuterm indexes***
- ***K-gram indexes***

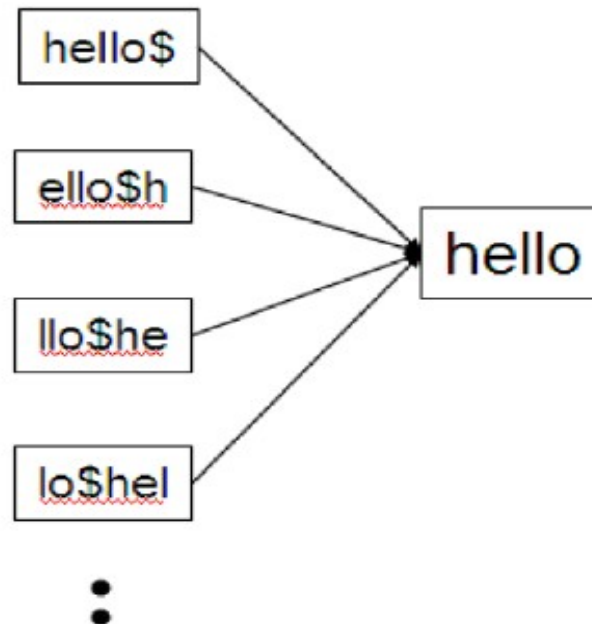
Both techniques share a common strategy:

- ***Express the given wildcard query q_w as a Boolean query Q on a specially constructed index, such that the answer to Q is a superset of the set of vocabulary terms matching q_w .***
- ***Then, we check each term in the answer to Q against q_w , discarding those vocabulary terms that do not match q_w .***

Permuterm indexes

- ***Introduce a special symbol \$ into the character set, to mark the end of a term***
- ***For term hello, index under:
hello\$, ello\$h, llo\$he, lo\$hel, o\$hell, \$hello***
where \$ is a special symbol.

Permuterm indexes



► Figure 3.3 A portion of a permuterm index.

- From the permuterm we can get the term and then from the standard index we can get the documents containing the term.

Permuterm indexes



\$hello

hello\$

ello\$h

llo\$he

lo\$hel

o\$hell



hello

\$help

help\$

elp\$h

lp\$he

p\$hel



help

Permuterm indexes



- *Consider the wildcard query $m*n$.*
- *Rotate this wildcard query so that the $*$ symbol appears at the end of the string after adding the $\$$ symbol*
- *Rotated wildcard query becomes $n\$m*$*
- *Look up this string in the permuterm index and get the terms*
- *Search in the inverted index all the documents indexed by these terms*
- *$M*n\$, *n\$m, n\$m*$*
- *$H*o, o\$h*$*

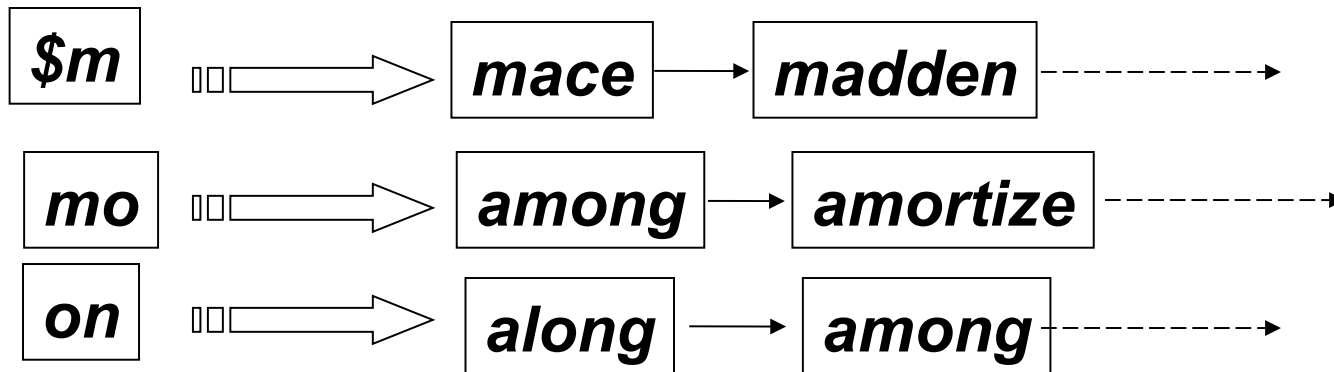
k-gram indexes



- ***A k-gram is a sequence of k characters.***
- ***cas, ast and stl are all 3-grams occurring in the term castle.***
- ***The full set of 3-grams generated for castle is: \$ca, cas, ast, stl, tle, le\$***
- ***In a k-gram index, the dictionary contains all k-grams that occur in any term in the vocabulary***
- ***Each postings list points from a k-gram to all vocabulary terms containing that k***

k-gram indexes

- *The k-gram index finds terms based on a query consisting of k-grams (here k=2).*



- Maintain a second inverted index from bigrams to dictionary terms that match each bigram.

k-gram indexes

- Query *mon** can now be run as
 - *\$m AND mo AND on*
- Gets terms that match AND version of the wildcard query
- But the enumerated term *moon* will also be retrieved (False Positive)
- Must post-filter these terms against query
- Surviving enumerated terms are then looked up in the term-document inverted index
- Fast, space efficient.
- *mo*

Exercise - Permuterm Index



Exercise 3.2

Write down the entries in the permuterm index dictionary that are generated by the term *mama*.

Exercise 3.3

If you wanted to search for *s*ng* in a permuterm wildcard index, what key(s) would one do the lookup on?

Tolerant Retrieval



- Wild-card queries
- **Spelling correction**
- Phonetic correction

Spelling correction



- *Among the various alternative correct spellings for a misspelled query, choose the “nearest” one.*
- *When two correctly spelled queries are tied (or nearly tied), select the one that is more common.*

Two forms of Correction:

– Isolated word

Check each word on its own for misspelling

Will not catch typos resulting in correctly spelled words

e.g., *from* → *form*

– Context-sensitive

Look at surrounding words,

e.g., *I flew form Heathrow to Narita*

Isolated word correction

- ***Fundamental premise – there is a lexicon from which the correct spellings come***
- ***Two basic choices for this:***
 - A standard lexicon such as
Webster's English Dictionary
An “industry-specific” lexicon – hand-maintained
 - The lexicon of the indexed corpus
E.g., all words on the web
All names, acronyms etc.

Isolated word correction

- ***Given a lexicon and a character sequence Q , return the words in the lexicon closest to Q***

Edit distance (Levenshtein distance)

Weighted edit distance

k

Edit distance

- *Given two strings S_1 and S_2 , the minimum number of operations to convert one to the other*
- *Operations are typically character-level*
 - Insert, Delete, Replace, (Transposition)
- *E.g., the edit distance from dof to dog is 1*
 - From **cat** to **act** is 2 (Just 1 with transpose.)
 - from **cat** to **dog** is 3.
-

Levenshtein Distance: Algorithm



Note: To understand this algorithm, Dynamic programming design technique understanding is required.

- The algorithm fills the entries in a matrix m whose two dimensions equals the length of the two strings, whose edit distances are being computed**
- The (i, j) entry of the matrix will hold the edit distance between the strings consisting of the first i characters of s_1 and the first j characters of s**

Levenshtein Distance: Algorithm

LEVENSHTEINDISTANCE(s_1, s_2)

```
1  for  $i \leftarrow 0$  to  $|s_1|$ 
2  do  $m[i, 0] = i$ 
3  for  $j \leftarrow 0$  to  $|s_2|$ 
4  do  $m[0, j] = j$ 
5  for  $i \leftarrow 1$  to  $|s_1|$ 
6  do for  $j \leftarrow 1$  to  $|s_2|$ 
7      do if  $s_1[i] = s_2[j]$ 
8          then  $m[i, j] = \min\{m[i-1, j]+1, m[i, j-1]+1, m[i-1, j-1]\}$ 
9          else  $m[i, j] = \min\{m[i-1, j]+1, m[i, j-1]+1, m[i-1, j-1]+1\}$ 
10 return  $m[|s_1|, |s_2|]$ 
```

Operations: insert (cost 1), delete (cost 1), replace (cost 1), copy (cost 0)

Each cell of Levenshtein matrix

Cost of getting here from my upper left neighbour (by copy or replace)	Cost of getting here from my upper neighbour (by delete)
Cost of getting here from my left neighbour (by insert)	Minimum cost out of these

Levenshtein Distance: Example

		f	a	s	t
	0	1	2	3	4
c	1	1	2	3	4
a	2	2	1	2	3
t	3	3	2	2	2
s	4	4	3	2	3

Weighted edit distance

- ***Same as edit distance, but the weight of an operation depends on the character(s) involved***

Meant to capture OCR or keyboard errors

Example: ***m*** more likely to be mis-typed as ***n*** than as ***q***

Therefore, replacing ***m*** by ***n*** is a smaller edit distance than by ***q***

This may be formulated as a probability model

- ***Requires weight matrix as input***

-

Using edit distances



- ***Given query, first enumerate all character sequences within a preset edit distance (e.g., 2)***
- ***Intersect this set with list of “correct” words***
- ***Show terms you found to user as suggestions***
- ***Alternatively,***
 - look up all possible corrections in the inverted index and return all docs -- slow
 - run with a single most likely correction

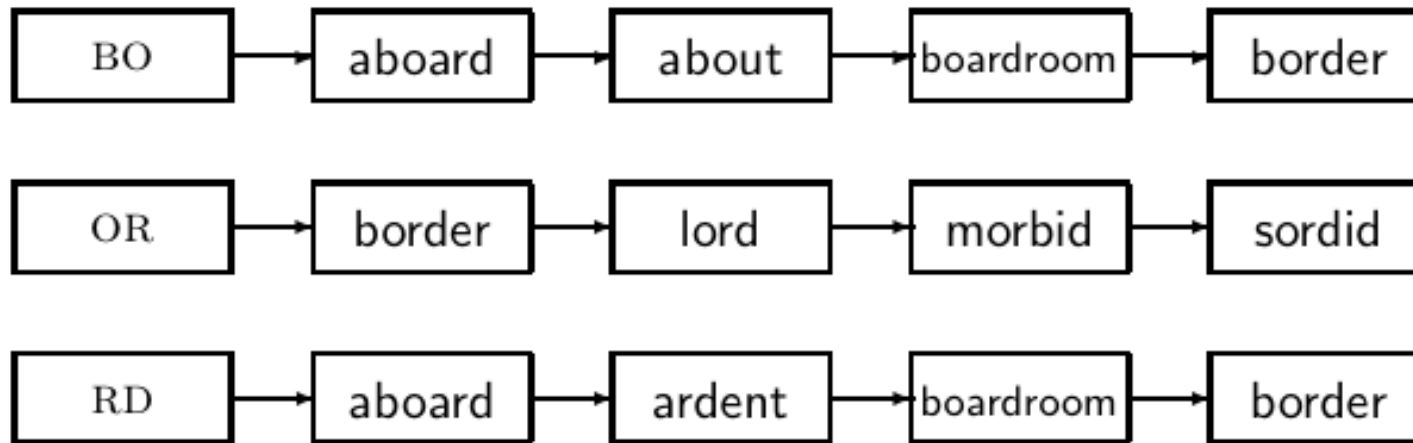
Using edit distances

- ***Given a (mis-spelled) query – do we compute its edit distance to every dictionary term?***
 - Expensive and slow
 - Alternative?
- ***One possibility is to use ***k*-gram overlap.*****
 -

k-gram index

- Enumerate all the k-grams in the query string as well as in the document
- Use the k-gram index to retrieve all terms matching any of the query k-grams
- Threshold by number of matching k-grams
- Example: bigram index, misspelled word **boardroom** as **bordroom**
- Bigrams: *bo, or, rd, dr, ro, oo, om*

k-gram index



k-gram index



Example with trigrams

- ***Suppose the text is November***
 - Trigrams are **nov**, **ove**, **vem**, **emb**, **mbe**, **ber**.
- ***The query is december***
 - Trigrams are **dec**, **ece**, **cem**, **emb**, **mbe**, **ber**.
- ***So 3 trigrams overlap (of 6 in each term)***
 -

k-gram index



- ***Jaccard coefficient***
- ***A commonly-used measure of overlap***
- ***Let X and Y be two sets; then the J.C. is $X \cap Y / X \cup Y$***
- ***Equals 1 when X and Y have the same elements and zero when they are disjoint***
- ***X and Y don't have to be of the same size***
- ***Always assigns a number between 0 and 1***
 - Now threshold to decide if you have a match
 -

Context-sensitive spell correction



- Take the Example : *flew **form** munich*
 - How can we correct **form** here?
- One idea: **hit-based** spelling correction
 - Retrieve “correct” terms close to each query term *for “flew form munich”*:
 - *flea for flew, from for form, munch for munich*
 - Now try all possible resulting phrases as queries with one word “fixed” at a time
 - Try query *“flea form munich”*
 - Try query *“flew from munich”*
 - Try query *“flew form munch”*
- The correct query *“flew from munich”* has the most hits.

Context-sensitive spell correction



Another approach

- ***Break phrase query into a conjunction of biwords.***
- ***Look for biwords that need only one term corrected.***
- ***Enumerate phrase matches and rank them.***

Tolerant Retrieval



- Wild-card queries
- Spelling correction
- **Phonetic correction**

Phonetic correction



- *Misspellings that arise because the user types a query that sounds like the target term*
- *The main idea here is to generate, for each term, a “phonetic hash” so that similar-sounding terms hash to the same value*
- *Algorithms for such phonetic hashing are collectively known as **soundex***

Soundex

- ***Class of heuristics to expand a query into **phonetic** equivalents***
 - Language specific – mainly for names
 - E.g., ***chebyshev*** → ***tchebycheff***

Soundex – algorithm

- *Turn every token to be indexed into a 4-character reduced form. Build an inverted index from these reduced forms to the original terms; call this the soundex index*
- *Do the same with query terms*
-

Soundex – algorithm

- 1. Retain the first letter of the word.***
- 2. Change all occurrences of the following letters to '0' (zero):***
'A', 'E', 'I', 'O', 'U', 'H', 'W', 'Y'.
- 3. Change letters to digits as follows:***
B, F, P, V → 1
C, G, J, K, Q, S, X, Z → 2
D, T → 3
L → 4
M, N → 5

R → 6

Soundex – algorithm

- 4. Remove one out of each pair of consecutive digits.***
- 5. Remove all zeros from the resulting string.***
- 6. Pad the resulting string with trailing zeros and return the first four positions, which will be of the form <uppercase letter> <digit> <digit> <digit>.***

E.g., Herman becomes H655.

What will be the soundex code for Hermann,Horman?

H065055

H06505—H655

Smith-Smyth Ericson-Eriksen-Erikson

Soundex



- *Soundex is the classic algorithm, provided by most databases (Oracle, Microsoft, ...)*
- *Not much useful for information retrieval*
- *Okay for “high recall” tasks (e.g., Interpol), though biased to names of certain nationalities*

Exercise

- ***Draw yourself a diagram showing the various indexes in a search engine incorporating all the functionality we have talked about***
- ***Identify some of the key design choices in the index pipeline:***
 - Does stemming happen before the Soundex index?
 - What about n -grams?
- ***Given a query, how would you parse and dispatch sub-queries to the various indexes?***

Resources



[Wild card search in Google](#)

[Edit distance demo](#)

[Wild card search in Google –Article](#)

[Soundex Demo1](#)

[Soundex Demo2](#)

Resources



Efficient spell retrieval:

K. Kukich. Techniques for automatically correcting words in text. ACM Computing Surveys 24(4), Dec 1992.

J. Zobel and P. Dart. Finding approximate matches in large lexicons. Software - practice and experience 25(3), March 1995. <http://citeseer.ist.psu.edu/zobel95finding.html>

Mikael Tillenius: Efficient Generation and Ranking of Spelling Error Corrections. Master's thesis at Sweden's Royal Institute of Technology. <http://citeseer.ist.psu.edu/179155.html>

Nice, easy reading on spell correction:

***Peter Norvig: How to write a spelling corrector [h
ttp://norvig.com/spell-correct.html](http://norvig.com/spell-correct.html)***



BITS Pilani
Pilani Campus

AIMLCZG537/DSECLZG537 Information Retrieval

Dr. Vijayalakshmi anand
BITS Pilani

Vector Space Model

Slides are adapted from Introduction to Information
Retrieval; Manning, Raghavan
<http://nlp.stanford.edu/IR-book>

Vector Space Model



- Term frequency and weighting
- The vector space model for scoring

Ranked retrieval

- Thus far, our queries have all been Boolean.
 - Documents either match or don't.
- Good for expert users with precise understanding of their needs and the collection.
- Not good for the majority of users.
 - Most users are incapable of writing Boolean queries (or they are, but they think it's too much work).
 - Most users don't want to wade through 1000s of results.
 - This is particularly true of web search.



Ranked retrieval models

- Rather than a set of documents satisfying a query expression, in **ranked retrieval**, the system returns an ordering over the (top) documents in the collection for a query
- When a system produces a ranked result set,
 - The size of the result set is not an issue
 - Just show the top k (≈ 10) results
 - Don't overwhelm the user

Ranked retrieval models

- Return in order the documents most likely to be useful to the searcher
- How can we rank-order the documents in the collection with respect to a query?
- Assign a score – say in $[0, 1]$ – to each document
- This score measures how well document and query “match”.

Query-document matching scores

- We need a way of assigning a score to a query/document pair
- **Let's start with a one-term query**
- If the query term does not occur in the document: score should be 0
- **The more frequent the query term in the document, the higher the score (should be)**
- We will look at a number of alternatives for this.

Jaccard coefficient

- A commonly used measure of overlap of two sets A and B
- $\text{jaccard}(A, B) = |A \cap B| / |A \cup B|$
- $\text{jaccard}(A, A) = 1$
- $\text{jaccard}(A, B) = 0$ if $A \cap B = 0$
- A and B don't have to be the same size.
- Always assigns a number between 0 and 1.

Jaccard coefficient

$$\text{jaccard}(A,B) = |A \cap B| / |A \cup B|$$

- Larger the term overlap of the two sets A and B , the better.
- Larger the *proportion* of term overlap with respect to the sizes of the sets A and B , the better.

Jaccard coefficient: Scoring example

- What is the query-document match score that the Jaccard coefficient computes for each of the two documents below?

Query: *ides of march*

Document 1: *caesar died in march*

Document 2: *the long march march*

Document 3: *the long march end april*

- $JACCARD(q, d1) = 1/6$
- $Jaccard(q, d2) = 1/5$

Issues with Jaccard for scoring

- It doesn't consider *term frequency* (how many times a term occurs in a document)
- Rare terms in a collection are more informative than frequent terms. Jaccard doesn't consider this information
- We need a more sophisticated way of normalizing the length of document

Term-document incidence matrix

	Antony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth
Antony	1	1	0	0	0	1
Brutus	1	1	0	1	0	0
Caesar	1	1	0	1	1	1
Calpurnia	0	1	0	0	0	0
Cleopatra	1	0	0	0	0	0
mercy	1	0	1	1	1	1
worser	1	0	1	1	1	0

Each document is represented by a binary vector $\in \{0,1\}^{|V|}$

Term-document count matrices

- Consider the number of occurrences of a term in a document:
 - Each document is a count vector

	Antony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth
Antony	157	73	0	0	0	1
Brutus	4	157	0	1	0	0
Caesar	232	227	0	2	1	1
Calpurnia	0	10	0	0	0	0
Cleopatra	57	0	0	0	0	0
mercy	2	0	3	5	5	1
worser	2	0	1	1	1	0

Bag of words model

- Vector representation doesn't consider the ordering of words in a document
- *John is quicker than Mary* and *Mary is quicker than John* have the same vectors
- This is called the bag of words model.
- In a sense, this is a step back: The positional index was able to distinguish these two documents.
- We will look at “recovering” positional information later.
- For now: bag of words model

Term frequency - tf

- The term frequency $tf_{t,d}$ of term t in document d is defined as the number of times that t occurs in d .
- We want to use tf when computing query-document match scores.
- Raw term frequency is not what we want:
 - A document with 10 occurrences of the term is more relevant than a document with 1 occurrence of the term.
 - But not 10 times more relevant.
- Relevance does not increase proportionally with term frequency.

Example



- Compute the **Jaccard** matching score and the **tf** matching score for the following query-document pairs.
 - a) q: [information on cars] d: “all you have ever wanted to know about cars”
 - b) q: [information on cars] d: “information on trucks, information on planes, information on trains”
 - c) q: [red cars and trucks] d: “cops stop red cars more often”

Log-frequency weighting

- The log frequency weight of term **t in d** is

$$w_{t,d} = \begin{cases} 1 + \log_{10} \text{tf}_{t,d} & \text{if } \text{tf}_{t,d} > 0 \\ 0 & \text{otherwise} \end{cases}$$

- $\text{tf}_{t,d} \rightarrow w_{t,d}$:

0 → 0, 1 → 1, 2 → 1.3, 10 → 2, 1000 → 4, etc.

- Score for a document-query pair: sum over terms t in both q and d :

$$\text{tf-matching-score}(q, d) = \sum_{t \in q \cap d} (1 + \log \text{tf}_{t,d})$$

- The score is 0 if none of the query terms is present in the document.

Document Frequency

- The document frequency is the **number of documents in the collection that the term t occurs in.**
- Use the frequency of the term in the collection for weighting and ranking.

Collection vs. Document frequency

- The collection frequency of t is the number of occurrences of t in the collection, counting multiple occurrences.

Example:

Word	Collection frequency	Document frequency
<i>insurance</i>	10440	3997
<i>try</i>	10422	8760

Document Frequency

- Rare terms are more informative than frequent terms.
- Consider a term in the query that is rare in the collection
- (e.g., arachnocentric).
- A document containing this term is very likely to be relevant.
- We want high weights for rare terms like arachnocentric

idf weight



- df_t is the document frequency, the number of documents that t occurs in.
- df_t is an inverse measure of the informativeness of term t .
- We define the **idf** weight of term t as follows:
 - $i df_t = \log (N / df_t)$
(N is the number of documents in the collection.)
- idf_t is a measure of the informativeness of the term.

idf example

suppose $N = 1,000,000$

term	df_t	idf_t
calpurnia	1	
animal	100	
sunday	1,000	
fly	10,000	
under	100,000	
the	1,000,000	

$$idf_t = \log_{10} (N/df_t)$$

Effect of idf on ranking

- Does idf have an effect on ranking for one-term queries, like
 - iPhone
- idf has no effect on ranking one term queries
 - idf affects the ranking of documents for queries with at least two terms
 - For the query **capricious person**, idf weighting makes occurrences of **capricious** count for much more in the final document ranking than occurrences of **person**.

tf-idf weighting

- The tf-idf weight of a term is the product of its **tf** weight and its **idf** weight.

$$w_{t,d} = (1 + \log \text{tf}_{t,d}) \cdot \log \frac{N}{\text{df}_t}$$

- Best known weighting scheme in information retrieval
 - Note: the “-” in tf-idf is a hyphen, not a minus sign.
 - Alternative names: tf.idf, tf x idf
- Increases with the number of occurrences within a document (term frequency)
- Increases with the rarity of the term in the collection (inverse document frequency)

Example



	Doc1	Doc2	Doc3
car	27	4	24
auto	3	33	0
insurance	0	33	29
best	14	0	17

term	df_t	idf_t
car	18,165	1.65
auto	6723	2.08
insurance	19,241	1.62
best	25,235	1.5

Exercise 6.10

Consider the table of term frequencies for 3 documents denoted Doc1, Doc2, Doc3 in Figure 6.9. Compute the tf-idf weights for the terms car, auto, insurance, best, for each document, using the idf values from Figure 6.8.

tf – idf for (cars, doc1) = $(1 + \log 27) * 1.65 = 2.43 * 1.65 = 4$

tf – idf for (cars, doc2) = $(1 + \log 4) * 1.65 = 2.64$

tf – idf for (cars, doc2) = $(1 + \log 24) * 1.65 = 3.93$

tf – idf score for term **cars** = $4 + 2.64 + 3.93 = 10.57$

Similar manner the scores for other terms has to be computed.

Example - Tf-idf scores



Tf-idf computation without taking logarithmic tf (raw tf value)

terms	Doc1	Doc2	Doc3

Car	44.55	6.6	39.6
Auto	6.24	68.64	0
Insurance	0	53.46	46.98
Best	21	0	25.5

Score for a document given a query

$$\text{Score}(q, d) = \sum_{t \in q \cap d} \text{tf.idf}_{t,d}$$

Binary $\rightarrow$ count $\rightarrow$ weight matrix

	Antony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth
Antony	5.25	3.18	0	0	0	0.35
Brutus	1.21	6.1	0	1	0	0
Caesar	8.59	2.54	0	1.51	0.25	0
Calpurnia	0	1.54	0	0	0	0
Cleopatra	2.85	0	0	0	0	0
mercy	1.51	0	1.9	0.12	5.25	0.88
worser	1.37	0	0.11	4.15	0.25	1.95

Each document is now represented by a real-valued vector of tf-idf weights $\in \mathbb{R}^M$

Documents as Vectors



- Each document is now represented by a real-valued vector of tf-idf weights $\in \mathbb{R}^{|V|}$
- It is a $|V|$ -dimensional real-valued vector space.
- Terms are axes of the space.
- Documents are points or vectors in this space.
- Very high-dimensional: tens of millions of dimensions when you apply this to web search engines
- Each vector is very sparse - most entries are zero.

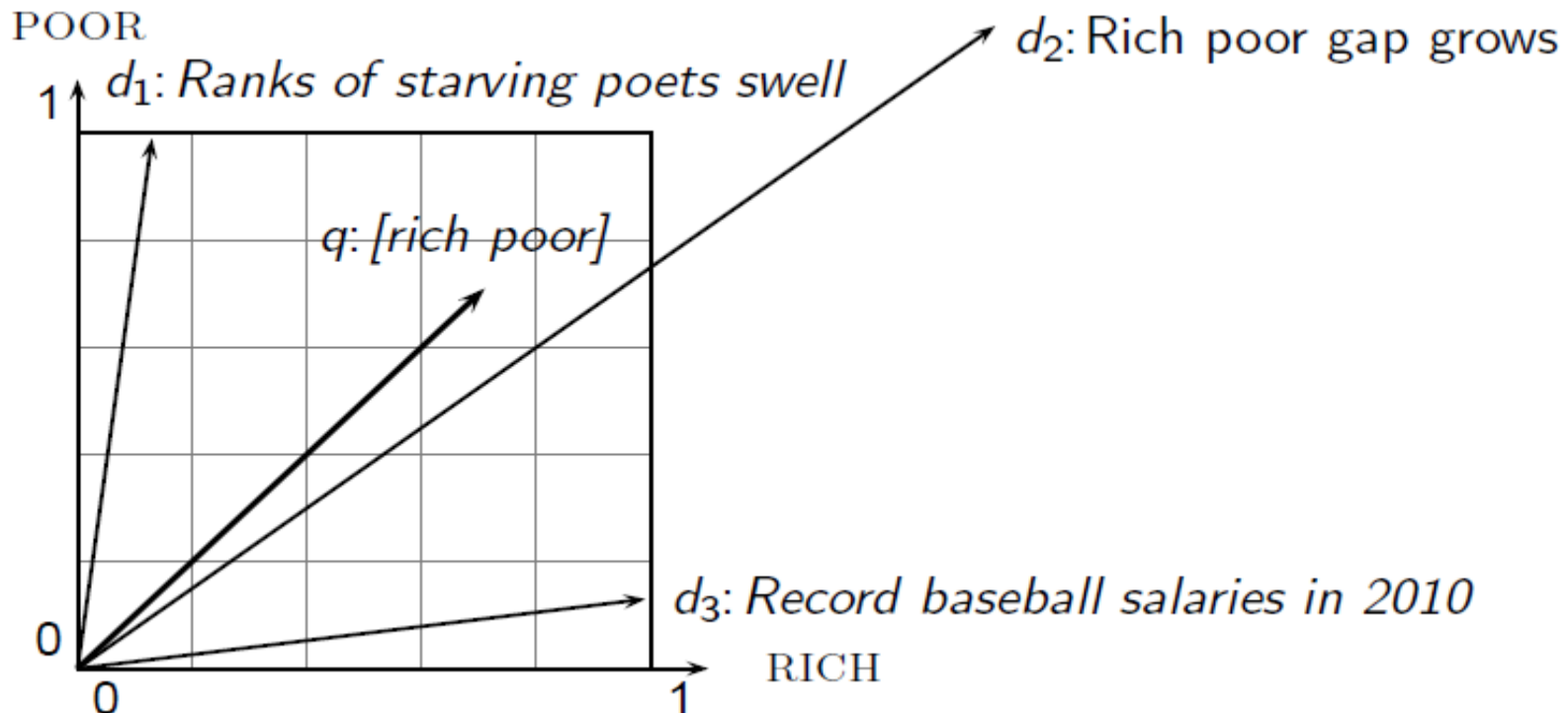
Queries as vectors

- Key idea 1: Do the same for queries: represent them as vectors in the space
- Key idea 2: Rank documents according to their proximity to the query in this space
- proximity = similarity of vectors
- proximity $\approx$ inverse of distance
- How to quantify the similarity between the vectors?

Similarity using distance(difference)



- The Euclidean distance between q and d_2 is large even though the distribution of terms in the query q and the distribution of terms in the document d_2 are very similar.





Use angle instead of distance

- Take a document d and append it to itself. Call this document d' .
- “Semantically” d and d' have the same content
- The Euclidean distance between the two documents can be quite large
- The angle between the two documents is 0, corresponding to maximal similarity.
- **Key idea: Rank documents according to angle with query.**

From angles to cosines

- The following two notions are equivalent.
 - Rank documents in increasing order of the angle between query and document
 - Rank documents in decreasing order of cosine angle between query and document
 - Cosine is a monotonically decreasing function for the interval $[0^\circ, 180^\circ]$

Cosine(query, document)

Dot product

$$\cos(\vec{q}, \vec{d}) = \frac{\vec{q} \bullet \vec{d}}{|\vec{q}| |\vec{d}|} = \frac{\vec{q}}{|\vec{q}|} \bullet \frac{\vec{d}}{|\vec{d}|} = \frac{\sum_{i=1}^{|\mathcal{V}|} q_i d_i}{\sqrt{\sum_{i=1}^{|\mathcal{V}|} q_i^2} \sqrt{\sum_{i=1}^{|\mathcal{V}|} d_i^2}}$$

q_i is the tf-idf weight of term i in the query

d_i is the tf-idf weight of term i in the document

$\cos(q, d)$ is the cosine similarity of q and d ... or, equivalently, the cosine of the angle between q and d .

Cosine for length-normalized vectors



- For length-normalized vectors, cosine similarity is simply the dot product (or scalar product):

$$\cos(\vec{q}, \vec{d}) = \vec{q} \bullet \vec{d} = \sum_{i=1}^{|\mathcal{V}|} q_i d_i$$

- for q, d length-normalized.

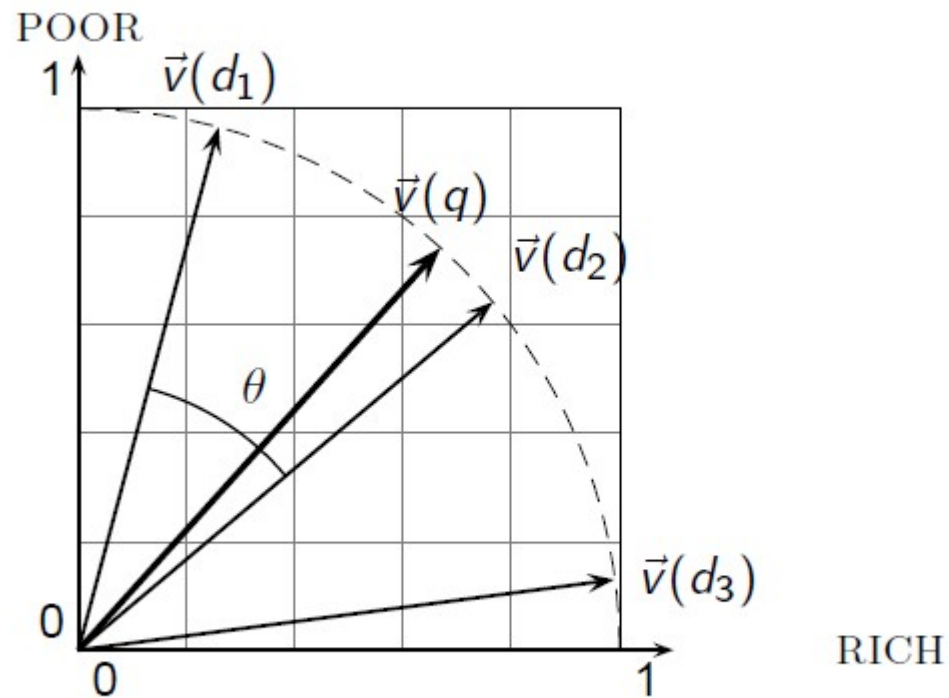
Length normalization

- A vector can be (length-) normalized by dividing each of its components by its length – for this we use the L_2 norm:

$$\|\vec{x}\|_2 = \sqrt{\sum_i x_i^2}$$

- Dividing a vector by its L_2 norm makes it a unit (length) vector.
- Effect on the two documents d and d' (d appended to itself) from earlier slide: they have identical vectors after length-normalization.
 - Long and short documents now have comparable weights

Cosine similarity



Cosine similarity among 3 documents



How similar are the novels?

SaS: Sense and Sensibility , **PaP: Pride and Prejudice**, and
WH: Wuthering Heights

term	SaS	PaP	WH
affection	115	58	20
jealous	10	7	11
gossip	2	0	6
wuthering	0	0	38

Term frequencies (counts)

Note: To simplify this example, we don't do idf weighting.

Cosine similarity among 3 documents



Log frequency weighting

term	SaS	PaP	WH
affection	3.06	2.76	2.30
jealous	2.00	1.85	2.04
gossip	1.30	0	1.78
wuthering	0	0	2.58

After length normalization

term	SaS	PaP	WH
affection	0.789	0.832	0.524
jealous	0.515	0.555	0.465
gossip	0.335	0	0.405
wuthering	0	0	0.588

$\cos(\text{SaS}, \text{PaP}) \approx$

$0.789 \times 0.832 + 0.515 \times 0.555 + 0.335 \times 0.0 + 0.0 \times 0.0$

≈ 0.94

$\cos(\text{SaS}, \text{WH}) \approx 0.79$

$\cos(\text{PaP}, \text{WH}) \approx 0.69$

$$3.02^2 + 2^2 + 1.30^2$$
$$= \text{SQRT}(15.05) = 3.87$$

$$3.06/3.87 = 0.79$$

Computing cosine scores

COSINESCORE(q)

```
1  float Scores[ $N$ ] = 0
2  float Length[ $N$ ]
3  for each query term  $t$ 
4  do calculate  $w_{t,q}$  and fetch postings list for  $t$ 
5      for each pair( $d, tf_{t,d}$ ) in postings list
6      do  $Scores[d] + = w_{t,d} \times w_{t,q}$ 
7  Read the array Length
8  for each  $d$ 
9  do  $Scores[d] = Scores[d] / Length[d]$ 
10 return Top  $K$  components of Scores[]
```

Weighting may differ in queries vs. documents



- We often use different weightings for queries and documents.
- Notation: ddd.qqq
- Example: Inc.ltn
- document: logarithmic tf, no df weighting, cosine normalization
- query: logarithmic tf, idf, no normalization
- **Example query: “best car insurance”**
- **Example document: “car insurance auto insurance”**

Example (Inc.Itn)



Query

Document

Word	tf-raw	tf-wt	df	idf	wt	tf-raw	tf-wt	wt	n'lize	product
auto										
best										
car										
insurance										

Example



Query

Document

Word	tf-raw	tf-wt	df	idf	wt	tf-raw	tf-wt	wt	n'lize	product
auto	0					1				
best	1					0				
car	1					1				
insurance	1					2				

Example



Query

Document

Word	tf-raw	tf-wt	df	idf	wt	tf-raw	tf-wt	wt	n'lize	product
auto	0	0				1	1			
best	1	1				0	0			
car	1	1				1	1			
insurance	1	1				2	1.3			

Example



Query

Document

Word	tf-raw	tf-wt	df	idf	wt	tf-raw	tf-wt	wt	n'lize	product
auto	0	0	5000			1	1			
best	1	1	50000			0	0			
car	1	1	10000			1	1			
insurance	1	1	1000			2	1.3			

Example



Query

Document

Word	tf-raw	tf-wt	df	idf	wt	tf-raw	tf-wt	wt	n'lize	product
auto	0	0	5000	2.3		1	1			
best	1	1	50000	1.3		0	0			
car	1	1	10000	2.0		1	1			
insurance	1	1	1000	3.0		2	1.3			

Example



Query

Document

Word	tf-raw	tf-wt	df	idf	wt	tf-raw	tf-wt	wt	n'lize	product
auto	0	0	5000	2.3	0	1	1			
best	1	1	50000	1.3	1.3	0	0			
car	1	1	10000	2.0	2.0	1	1			
insurance	1	1	1000	3.0	3.0	2	1.3			

Example



$$\text{Doc length} = \sqrt{1^2 + 0^2 + 1^2 + 1.3^2} \approx 1.92$$

Query

Document

Word	tf-raw	tf-wt	df	idf	wt	tf-raw	tf-wt	wt	n'lize	product
auto	0	0	5000	2.3	0	1	1	1		
best	1	1	50000	1.3	1.3	0	0	0		
car	1	1	10000	2.0	2.0	1	1	1		
insurance	1	1	1000	3.0	3.0	2	1.3	1.3		

Example



$$\text{Doc length} = \sqrt{1^2 + 0^2 + 1^2 + 1.3^2} \approx 1.92$$

Query

Document

Word	tf-raw	tf-wt	df	idf	wt	tf-raw	tf-wt	wt	n'lize	product
auto	0	0	5000	2.3	0	1	1	1	0.52	
best	1	1	50000	1.3	1.3	0	0	0	0	
car	1	1	10000	2.0	2.0	1	1	1	0.52	
insurance	1	1	1000	3.0	3.0	2	1.3	1.3	0.68	

Example



$$\text{Doc length} = \sqrt{1^2 + 0^2 + 1^2 + 1.3^2} \approx 1.92$$

Query

Document

Word	tf-raw	tf-wt	df	idf	wt	tf-raw	tf-wt	wt	n'lize	product
auto	0	0	5000	2.3	0	1	1	1	0.52	0
best	1	1	50000	1.3	1.3	0	0	0	0	0
car	1	1	10000	2.0	2.0	1	1	1	0.52	1.04
insurance	1	1	1000	3.0	3.0	2	1.3	1.3	0.68	2.04

Final similarity score between query and document = 3.08

Example



- Reconstruct the Table in the previous slides with the weighting as **Inc.Itc**

Word	tf-raw	tf-wt	df	idf	wt	n'lize	tf-raw	tf-wt	wt	n'lize	product
auto	0	0	5000	2.3	0	0	1	1	1	0.52	0
best	1	1	50000	1.3	1.3	0.34	0	0	0	0	0
car	1	1	10000	2.0	2.0	0.52	1	1	1	0.52	0.27
insurance	1	1	1000	3.0	3.0	0.78	2	1.3	1.3	0.68	0.53

Exercise 6.19



- Compute the vector space similarity between the query “**digital cameras**” and the document “**digital cameras and video cameras**” by filling out the empty columns in Table 6.1. Assume $N = 10,000,000$, logarithmic term weighting for query and document, idf weighting for the query only and cosine normalization for the document only. Treat and as a stop word. Enter term counts in the tf columns. What is the final similarity score?

word	query					document			
	tf	wf	df	idf	$q_i = wf \cdot idf$	tf	wf	$d_i = \text{normalized wf}$	$q_i \cdot d_i$
digital			10,000						
video			100,000						
cameras			50,000						

► Table 6.1 Cosine computation for Exercise 6.19.

Exercise 6.19 – Solution

Word	Query					document			qi*di
	tf	wf	df	idf	qi=wf-idf	tf	wf	di=normalized wf	
digital	1	1	10,000	3	3	1	1	0.52	1.56
video	0	0	100,000	2	0	1	1	0.52	0
Cameras	1	1	50,000	2.3	2.3	2	1.3	0.68	1.56

Similarity score = $1.56 + 1.56 = 3.12$



Summary – vector space ranking

- Represent the query as a weighted tf-idf vector
- Represent each document as a weighted tf-idf vector
- Compute the cosine similarity score for the query vector and each document vector
- Rank documents with respect to the query by score
- Return the top K (e.g., $K = 10$) to the user

Problems with Vector Space Model

- Missing semantic information (e.g. word sense).
- Missing syntactic information (e.g. phrase structure, word order, proximity information).
- Assumption of term independence (e.g. ignores synonymy).
- Lacks the control of a Boolean model (e.g., requiring a term to appear in a document).
- Given a two-term query “A B”, may prefer a document containing A frequently but not B, over a document that contains both A and B, but both less frequently.

References



- Ranked Retrieval
<https://www.youtube.com/watch?v=CTwqDj5gtLo> 5
- Jaccard Score (14 mins)
<https://www.youtube.com/watch?v=eqpzVedX1kY&t=605s>
- TF and bag of words (24 mins)
https://www.youtube.com/watch?v=n_ul9GIOIkQ
- WDM 60: Inverse Document Frequency (16 mins)
<https://www.youtube.com/watch?v=97Ey6aCeask>
- WDM 61: TF-IDF Score (16 mins)
<https://www.youtube.com/watch?v=r8zeOv638-A>
- WDM 62: Documents AS TF-IDF Vectors (16 mins)
<https://www.youtube.com/watch?v=zUyWj7Bx8HM>
- WDM 63: Length Normalization (28 mins)
<https://www.youtube.com/watch?v=70n2xxgEk7w>
- Cosine Similarity Example (24 mins)
<https://www.youtube.com/watch?v=HW9W6EBytLg>
- [WDM 66: Variants of TF IDF Weights - YouTube](#)



BITS Pilani
Pilani Campus

AIMLCZG537/DSECLZG537 Information Retrieval

Dr. Vijayalakshmi Anand
BITS PILANI

Index construction

- T1, Chapter 4

Slides are adapted from **Introduction to Information Retrieval; Manning, Raghavan**

<http://nlp.stanford.edu/IR-book>

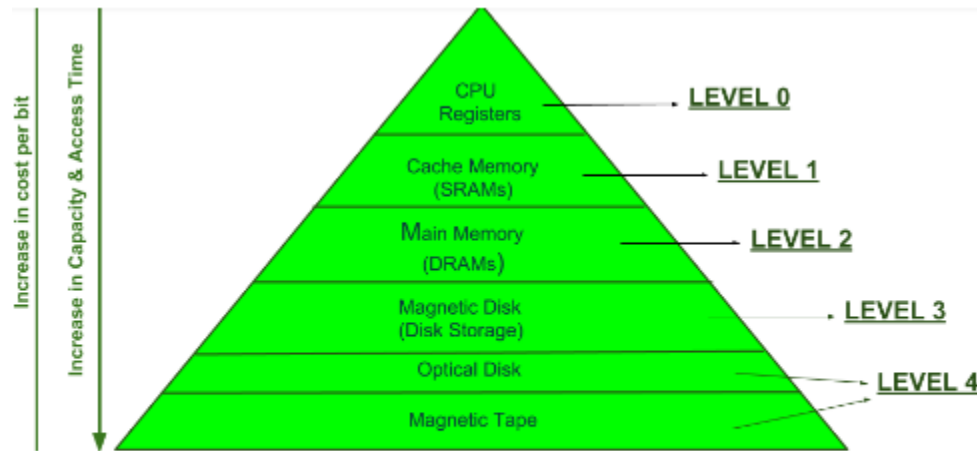
Chapter 4: Index construction

- Hardware Basics
- Blocked sort-based Indexing
- Single pass in-memory indexing
- Distributed and Dynamic indexing

Hardware Basics



- When building an information retrieval (IR) system, many decisions are based on the characteristics of the computer hardware on which the system runs.
 - Access to data in memory is ***much*** faster than access to data on disk. - Caching



MEMORY HIERARCHY DESIGN

- **Disk seeks:** No data is transferred from disk while the disk head is being positioned.
- Transferring one large chunk of data from disk to memory is faster than transferring many small chunks.
- **Disk I/O is block-based:** Reading and writing of entire blocks (as opposed to smaller chunks).
- Block sizes: 8KB to 256 KB.
- Compressed data read
- Servers used in IR systems typically have several gigabytes (GB) of main memory, sometimes tens of GB. Available disk space is several orders of magnitude larger

Blocked sort-based indexing



Reuters-RCV1:

- Shakespeare's collected works are not large enough to demonstrate scalable index construction algorithms.
- Instead, we will use the Reuters RCV1 collection.
- English newswire articles published in a 12-month period (1995/6)

A Reuters RCV1 Document

innovate

achieve

lead



You are here: [Home](#) > [News](#) > [Science](#) > [Article](#)

Go to a Section: [U.S.](#) [International](#) [Business](#) [Markets](#) [Politics](#) [Entertainment](#) [Technology](#) [Sports](#) [Oddly Enough](#)

Extreme conditions create rare Antarctic clouds

Tue Aug 1, 2006 3:20am ET

[Email This Article](#) | [Print This Article](#) | [Reprints](#)

[\[-\]](#) Text [\[+\]](#)



SYDNEY (Reuters) - Rare, mother-of-pearl colored clouds caused by extreme weather conditions above Antarctica are a possible indication of global warming, Australian scientists said on Tuesday.

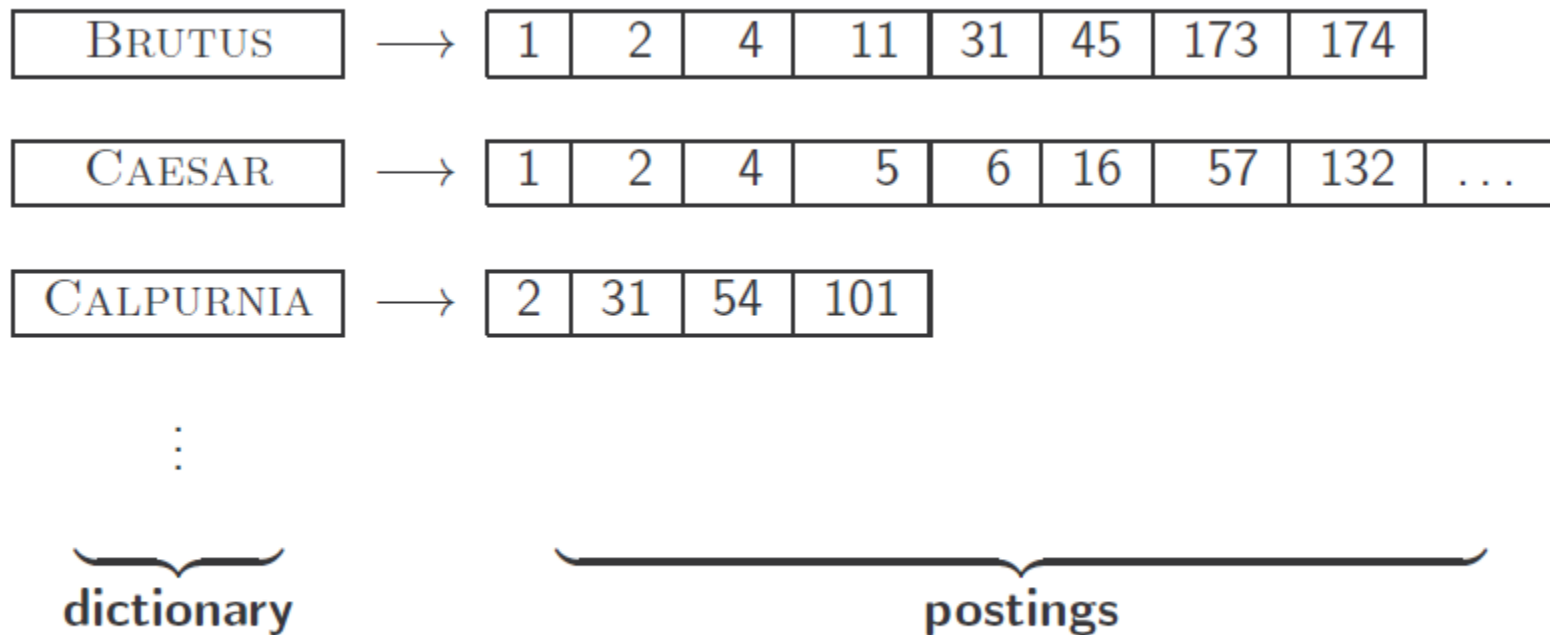
Known as nacreous clouds, the spectacular formations showing delicate wisps of colors were photographed in the sky over an Australian meteorological base at Mawson Station on July 25.

Reuters RCV1 Statistics



■ symbol	statistic	value
■ N	documents	800,000
■ L	avg. # tokens per doc	200
■ M	terms (= word types)	400,000
■	avg. # bytes per token (incl. spaces/punct.)	6
■	avg. # bytes per token (without spaces/punct.)	4.5
■	avg. # bytes per term	7.5
■	non-positional postings	100,000,000

Inverted Index construction



Inverted Index construction

term	docID		term	docID
I	1		ambitious	2
did	1		be	2
enact	1		brutus	1
julius	1		brutus	2
caesar	1		capitol	1
I	1		caesar	1
was	1		caesar	2
killed	1		caesar	2
i'	1		did	1
the	1		enact	1
capitol	1		hath	1
brutus	1		I	1
killed	1		I	1
me	1	⇒	i'	1
so	2		it	2
let	2		julius	1
it	2		killed	1
be	2		killed	1
with	2		let	2
caesar	2		me	1
the	2		noble	2
noble	2		so	2
brutus	2		the	1
hath	2		the	2
told	2		told	2
you	2		you	2
caesar	2		was	1
was	2		was	2
ambitious	2		with	2

Sort-based index construction

- As we build index, we parse docs one at a time.
- The final postings for any term are incomplete until the end.
- But for large collections, we cannot keep all postings in memory and do the sort in-memory at the end
- We cannot sort very large sets of records on disk either (too many disk seeks)
- Need to store intermediate results on disk.
- Need an external sorting algorithm.

Sort-based index construction

- We must sort $T = 100,000,000$ non-positional postings.
 - Each posting has size 12 bytes (4+4+4: termID, docID, term frequency).
- Define a block to consist of 10,000,000 such postings
 - Can easily fit that many postings into memory.
 - 10 such blocks for RCV1.
- Basic idea of algorithm:

For each block: (i) accumulate postings, (ii) sort in memory, (iii) write to disk
- Then merge the blocks into one long sorted order.

Blocked sort-based indexing (BSBI)



- Segments the collection into parts of equal size
- Sorts the termID – docID pair of each part in memory
- Stores intermediate sorted results on disk
- Merges all intermediate results into the final index.

BSBI Index Construction Algorithm



BSBIINDEXCONSTRUCTION()

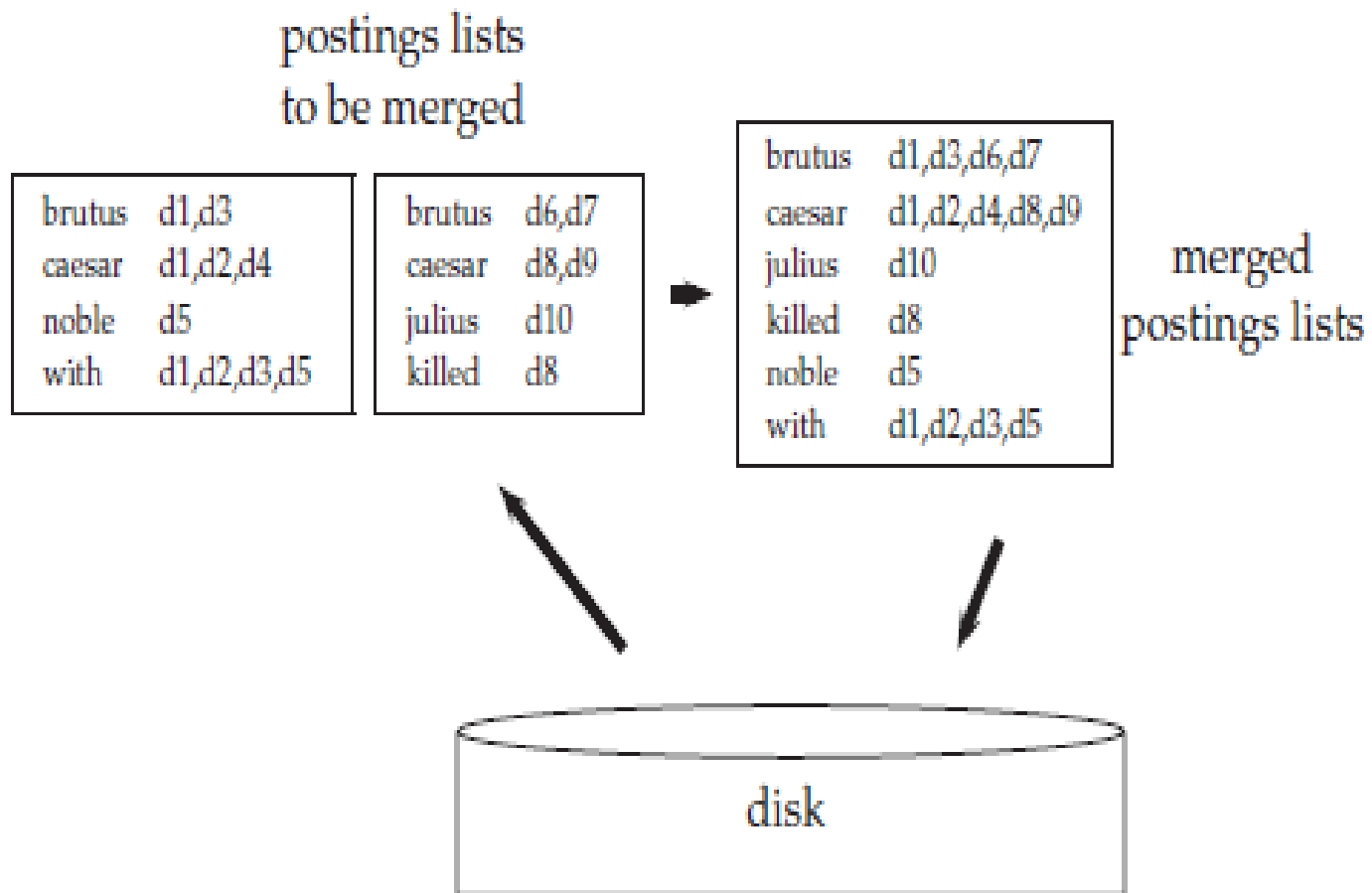
```
1   $n \leftarrow 0$ 
2  while (all documents have not been processed)
3  do  $n \leftarrow n + 1$ 
4       $block \leftarrow \text{PARSENEXTBLOCK}()$ 
5      BSBI-INVERT( $block$ )
6      WRITEBLOCKTODISK( $block, f_n$ )
7  MERGEBLOCKS( $f_1, \dots, f_n; f_{\text{merged}}$ )
```

BSBI Index Construction Algorithm



- **PARSENEXTBLOCK** --- The algorithm parses documents into termID–docID pairs and accumulates the pairs in memory until a block of a fixed size is full
- The block is then inverted and written to disk.
- Inversion involves two steps. First, sort the termID–docID pairs. Next, collect all termID–docID pairs with the same termID into a postings list, where a *posting* is simply a docID.

MERGEBLOCKS - Example



BSBI Algorithm



- **Excellent scaling properties**
- We need a data structure in order to implement a term to termID mapping (which grows dynamically)
- For very large collections, this data structure does not fit into memory
- We would end up with a scalable, but very slow index construction method
- More Scalable alternative is

Single-pass in-memory indexing (SPIMI)

Single-Pass In-Memory Indexing (SPIMI)



- SPIMI uses terms instead of termIDs
- Writes each block's dictionary to disk
- Starts a new dictionary for the next block
- Can index collections of any size as long as there is enough disk space available.

Single-Pass In-Memory Indexing (SPIMI)



- Key idea 1: Generate separate dictionaries for each block – no need to maintain term-termID mapping across blocks.
- Key idea 2: Don't sort. Accumulate postings in postings lists as they occur.

Single-Pass In-Memory Indexing (SPIMI)

innovate

achieve

lead

SPIMI-INVERT(*token_stream*)

```
1  output_file = NEWFILE()
2  dictionary = NEWHASH()
3  while (free memory available)
4  do token  $\leftarrow$  next(token_stream)
5      if term(token)  $\notin$  dictionary
6          then postings_list = ADDTODICTIONARY(dictionary, term(token))
7          else postings_list = GETPOSTINGSLIST(dictionary, term(token))
8      if full(postings_list)
9          then postings_list = DOUBLEPOSTINGSLIST(dictionary, term(token))
10     ADDTOPOSTINGSLIST(postings_list, docID(token))
11 sorted_terms  $\leftarrow$  SORTTERMS(dictionary)
12 WRITEBLOCKTODISK(sorted_terms, dictionary, output_file)
13 return output_file
```

Single-Pass In-Memory Indexing (SPIMI)



- Compression makes SPIMI even more efficient.
 - Compression of terms
 - Compression of posting

Distributed Indexing



- Data Collection is too big that index construction cannot be performed efficiently on a single machine
- Especially for web-scale indexing it must use a distributed computing cluster.
- Web search engines, therefore, use *distributed indexing* algorithms for index construction.
- The result of the construction process is a distributed index that is partitioned across several machines – either according to term or according to document.

Distributed Indexing



- We need large computer *clusters* to construct any reasonably sized web index.
- Clustering is to solve large computing problems on cheap commodity machines or *nodes* that are built from standard parts (processor, memory, disk)
- Individual machines are fault-prone - Can unpredictably slow down or fail

Google data centers

- Google data centers mainly contain commodity machines.
- Data centers are distributed around the world.
- Estimate: a total of 1 million servers, 3 million processors/cores (Gartner 2007)
- Estimate: Google installs 100,000 servers each quarter.
 - Based on expenditures of 200–250 million dollars per year
- This would be 10% of the computing capacity of the world.

[Google Data Centre Tour](#)

M45: Open Academic Cluster



Yahoo M45 cluster:

- Datacenter in a Box (DiB)
- 1000 nodes, 4000 cores, 3TB RAM,
- 1.5PB disk
- High bandwidth connection to Internet
- Located on Yahoo campus
- World's top 50 supercomputer

Large scale computing



Challenges:

- How do you distribute computation?
- Distributed/parallel programming is hard
- Machines fail

Map-reduce addresses all the above

- Google's computational/data manipulation model
- Elegant way to work with big data

Distributed Indexing



- The distributed index construction method described is an application of ***MapReduce***, a general architecture for distributed computing
- MapReduce is designed for large computer clusters.

Distributed indexing

- Maintain a *master* machine directing the indexing job.
- Break up indexing into sets of (parallel) tasks.
- Master machine assigns/reassigns each task to an idle machine from a pool.

MapReduce



Two Phases:

- Map and Reduce
 - split up the computing job into chunks that standard machines can process in a short time.

Steps in MapReduce Process:

- The input data, is split into n splits where the size of the split is chosen to ensure that the work can be distributed evenly and efficiently
- The *map phase* of MapReduce consists of mapping splits of the input data to key-value pairs.
- Each split is a subset of documents (corresponding to blocks in BSBI/SPIMI)

Map Phase-Parsers

- Master assigns a split to an idle parser machine
- Each parser writes its output to local intermediate files, the **segment files**
- Parser writes pairs into j partitions
- Each partition is for a range of terms' first letters
 - (e.g., **a-f**, **g-p**, **q-z**) – here $j = 3$.
 - Each term partition thus corresponds to r segments files, where r is the number of parsers

$j = 3$
par

a b c
g p
q-z

(a-z) → Partition (1-3)
gook → par (1-3)

Reduce phase - Inverters

- An inverter collects all (term,doc) pairs (= postings) for one term-partition
- Sorts and writes to postings lists.

due
to
care



MapReduce

- The index construction algorithm we just described is an instance of MapReduce
- MapReduce (Dean and Ghemawat 2004) is a robust and conceptually simple framework for distributed computing without having to write code for the distribution part
- They describe the Google indexing system (ca. 2002) as consisting of a number of phases, each implemented in MapReduce.

MapReduce

- Index construction was just one phase.
- Another phase: transforming a term-partitioned index into a document-partitioned index.
 - ***Term-partitioned:*** one machine handles a subrange of terms
 - ***Document-partitioned:*** one machine handles a subrange of documents
- Most search engines use a document-partitioned index, better load balancing, etc.

Schema for index construction in MapReduce

Schema of map and reduce functions

map: $\text{input} \rightarrow \text{list}(k, v)$ reduce: $(k, \text{list}(v)) \rightarrow \text{output}$

Instantiation of the schema for index construction

map: $\text{web collection} \rightarrow \text{list}(\text{termID}, \text{docID})$

reduce: $(\langle \text{termID1}, \text{list}(\text{docID}) \rangle, \langle \text{termID2}, \text{list}(\text{docID}) \rangle, \dots)$
 $\rightarrow (\text{postings list1}, \text{postings list2}, \dots)$

Example for index construction

map: $d2 : C \text{ died. } d1 : C \text{ came, } C \text{ c'ed.} \rightarrow (\langle C, d2 \rangle,$
 $\langle \text{died}, d2 \rangle, \langle C, d1 \rangle, \langle \text{came}, d1 \rangle, \langle C, d1 \rangle, \langle \text{c'ed}, d1 \rangle$

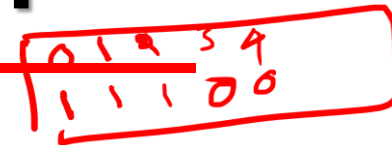
reduce: $(\langle C, (d2, d1, d1) \rangle, \langle \text{died}, (d2) \rangle, \langle \text{came}, (d1) \rangle,$
 $\langle \text{c'ed}, (d1) \rangle) \rightarrow (\langle C, (d1:2, d2:1) \rangle, \langle \text{died}, (d2:1) \rangle,$
 $\langle \text{came}, (d1:1) \rangle, \langle \text{c'ed}, (d1:1) \rangle)$

Dynamic Indexing



- So far, we have assumed that the document collection is static, fine for collections that change infrequently or never (e.g., the Bible or Shakespeare).
- Most collections are modified frequently with documents being added, deleted, and updated.
 - new terms need to be added to the dictionary,
 - postings lists need to be updated for existing terms.

Simplest approach



- Maintain “big” main index
- New docs go into “small” auxiliary index
- Search across both, merge results
- Deletions
 - Invalidation bit-vector for deleted docs
 - Filter docs output on a search result by this invalidation bit-vector
- Periodically, re-index into one main index

Issues with main and auxiliary indexes

- Problem of frequent merges
- Poor performance during merge
- Actually:
 - Merging of the auxiliary index into the main index is efficient if we keep a separate file for each postings list.
 - Merge is the same as a simple append.
 - But then we would need a lot of files – inefficient for O/S.
- Assumption : The index is one big file.
- In reality: Use a scheme somewhere in between (e.g., split very large postings lists, collect postings lists of length 1 in one file etc.)

Logarithmic merge

- Maintain a series of indexes, each twice as large as the previous one.
- Keep smallest (Z_0) in memory
- Larger ones ($I_0, I_1, \dots$) on disk
- If Z_0 gets too big ($> n$), write to disk as I_0
- or merge with I_0 (if I_0 already exists) as Z_1
- Either write merge Z_1 to disk as I_1 (if no I_1)
- Or merge with I_1 to form Z_2
- etc.

LMERGEADDTOKEN(*indexes*, Z_0 , *token*)

```

1   $Z_0 \leftarrow \text{MERGE}(Z_0, \{\text{token}\})$ 
2  if  $|Z_0| = n$ 
3      then for  $i \leftarrow 0$  to  $\infty$ 
4          do if  $l_i \in \text{indexes}$ 
5              then  $Z_{i+1} \leftarrow \text{MERGE}(l_i, Z_i)$ 
6                  ( $Z_{i+1}$  is a temporary index on disk.)
7                   $\text{indexes} \leftarrow \text{indexes} - \{l_i\}$ 
8              else  $l_i \leftarrow Z_i$     ( $Z_i$  becomes the permanent index  $l_i$ .)
9                   $\text{indexes} \leftarrow \text{indexes} \cup \{l_i\}$ 
10                     BREAK
11          $Z_0 \leftarrow \emptyset$ 

```

LOGARITHMICMERGE()

```

1   $Z_0 \leftarrow \emptyset$     ( $Z_0$  is the in-memory index.)
2   $\text{indexes} \leftarrow \emptyset$ 
3  while true
4  do LMERGEADDTOKEN(indexes,  $Z_0$ , GETNEXTTOKEN())

```

Further issues with multiple indexes

- Collection-wide statistics are hard to maintain
- E.g., when we spoke of spell-correction: which of several corrected alternatives do we present to the user?
 - We said, pick the one with the most hits
- How do we maintain the top ones with multiple indexes and invalidation bit vectors?
 - One possibility: ignore everything but the main index for such ordering

Dynamic indexing at search engines



- All the large search engines now do dynamic indexing
- Their indices have frequent incremental changes
 - News items, blogs, new topical web pages
- But (sometimes/typically) they also periodically reconstruct the index from scratch
 - Query processing is then switched to the new index, and the old index is then deleted

Other sorts of indexes

- **Positional indexes**
 - Same sort of sorting problem ... just larger
- **Building character n -gram indexes:**
 - As text is parsed, enumerate n -grams.
 - For each n -gram, need pointers to all dictionary terms containing it – the “postings”.
 - Note that the same “postings entry” will arise repeatedly in parsing the docs – need efficient hashing to keep track of this.
 - E.g., that the trigram uou occurs in the term **deciduous** will be discovered on each text occurrence of **deciduous**
 - Only need to process each term once

Exercise 0.46

For $n = 2$ and $1 \leq T \leq 30$, perform a step-by-step simulation of the algorithm in Figure 4.7. Create a table that shows, for each point in time at which $T = 2 * k$ tokens have been processed ($1 \leq k \leq 15$), which of the three indexes $I_0, \dots, I_3$ are in use. The first three lines of the table are given below.

	I_3	I_2	I_1	I_0
2	0	0	0	0
4	0	0	0	1
6	0	0	1	0

SOLUTION.

	I_3	I_2	I_1	I_0
2	0	0	0	0
4	0	0	0	1
6	0	0	1	0
8	0	0	1	1
10	0	1	0	0
12	0	1	0	1
14	0	1	1	0
16	0	1	1	1
18	1	0	0	0
20	1	0	0	1
22	1	0	1	0
24	1	0	1	1
26	1	1	0	0

Google Data Center 360 degree Tour

<https://www.sunbirddcim.com/video/google-data-center-360%C2%B0-tour>

Resources



Chapter 4 of IIR

Original publication on MapReduce: Dean and Ghemawat (2004)

Original publication on SPIMI: Heinz and Zobel (2003)

41-30 mins <https://www.youtube.com/watch?v=dH0pEySEoQo>

42-13 mins [WDM 42: Index Construction Using InMemory Sorting - YouTube](#)

43- 43 mins

44-12 mins(repeated part of 43) no need to watch 45-10 mins(repeated part of 43) no need to watch

46-25 mins (skip 8 mins) https://www.youtube.com/watch?v=WHS9f_wbU-E 47-QA of 46 not required [WDM 47: Q & A on Distributed Index – YouTube](#) 48 – 24 mins [WDM 48: Map Reduce – YouTube](#)

49 – 11 mins [WDM 49: Dynamic indexing using naive approach - YouTube](#) 50- 17 mins [WDM 50: Dynamic indexing using logarithmic merge – YouTube](#) 51- 3 minutes [WDM 51: Issues With Multiple Indexes - YouTube](#)



BITS Pilani
Pilani Campus

AIMLCZG537/DSECLZG537 Information Retrieval

Dr. Vijayalakshmi Anand
BITS -Pilani

Index Compression (T1, Chapter 5)

Slides are adapted from **Introduction to Information Retrieval; Manning, Raghavan**
<http://nlp.stanford.edu/IR-book>

Need for Compression

- **Use less disk space**
 - Saves a little money
- **Keep more stuff in memory**
 - Increases speed
- **Increase speed of data transfer from disk to memory**
 - [read compressed data | decompress] is faster than [read uncompressed data]
 - Premise: Decompression algorithms are fast
 - True of the decompression algorithms we use



Why compression for inverted indexes?

Dictionary

- Make it small enough to keep in main memory
- Make it so small that you can keep some postings lists in main memory too

Postings file(s)

- Reduce disk space needed
- Decrease time needed to read postings lists from disk
- Large search engines keep a significant part of the postings in memory.
 - Compression lets you keep more in memory
- We will devise various IR-specific compression schemes

Reuters-RCV1:

- Shakespeare's collected works are not large enough to demonstrate scalable index construction algorithms.
- Instead, we will use the Reuters RCV1 collection.
- English newswire articles published in a 12-month period (1995/6)

A Reuters RCV1 Document

innovate

achieve

lead



You are here: [Home](#) > [News](#) > [Science](#) > [Article](#)

Go to a Section: [U.S.](#) [International](#) [Business](#) [Markets](#) [Politics](#) [Entertainment](#) [Technology](#) [Sports](#) [Oddly Enough](#)

Extreme conditions create rare Antarctic clouds

Tue Aug 1, 2006 3:20am ET

[Email This Article](#) | [Print This Article](#) | [Reprints](#)

[\[-\]](#) Text [\[+\]](#)



SYDNEY (Reuters) - Rare, mother-of-pearl colored clouds caused by extreme weather conditions above Antarctica are a possible indication of global warming, Australian scientists said on Tuesday.

Known as nacreous clouds, the spectacular formations showing delicate wisps of colors were photographed in the sky over an Australian meteorological base at Mawson Station on July 25.

Reuters RCV1 Statistics



symbol	statistic	value
N	documents	800,000
L	avg. # tokens per doc	200
M	terms (= word types)	~400,000
	avg. # bytes per token (incl. spaces/punct.)	6
	avg. # bytes per token (without spaces/punct.)	4.5
	avg. # bytes per term	7.5
	non-positional postings	100,000,000

Index parameters vs. what we index (details //R Table 5.1)

size of	word types (terms)			non-positional postings			positional postings		
	dictionary			non-positional index			positional index		
	Size (K)	$\Delta\%$	cumul %	Size (K)	$\Delta\%$	cumul %	Size (K)	$\Delta\%$	cumul %
Unfiltered	484			109,971			197,879		
No numbers	474	-2	-2	100,680	-8	-8	179,158	-9	-9
Case folding	392	-17	-19	96,969	-3	-12	179,158	0	-9
30 stopwords	391	-0	-19	83,390	-14	-24	121,858	-31	-38
150 stopwords	391	-0	-19	67,002	-30	-39	94,517	-47	-52
stemming	322	-17	-33	63,812	-4	-42	94,517	0	-52



Lossless vs. lossy compression

Lossless compression: All information is preserved

- What we mostly do in IR

Lossy compression: Discard some information

- Several of the preprocessing steps can be viewed as lossy compression: case folding, stop words, stemming, number elimination.



Vocabulary vs. collection size

- How big is the term vocabulary?
 - That is, how many distinct words are there?
- Can we assume an upper bound?
 - **Can't** : At least $70^{20} = 10^{37}$ different words of length 20
- In practice, the vocabulary will keep growing with the collection size.

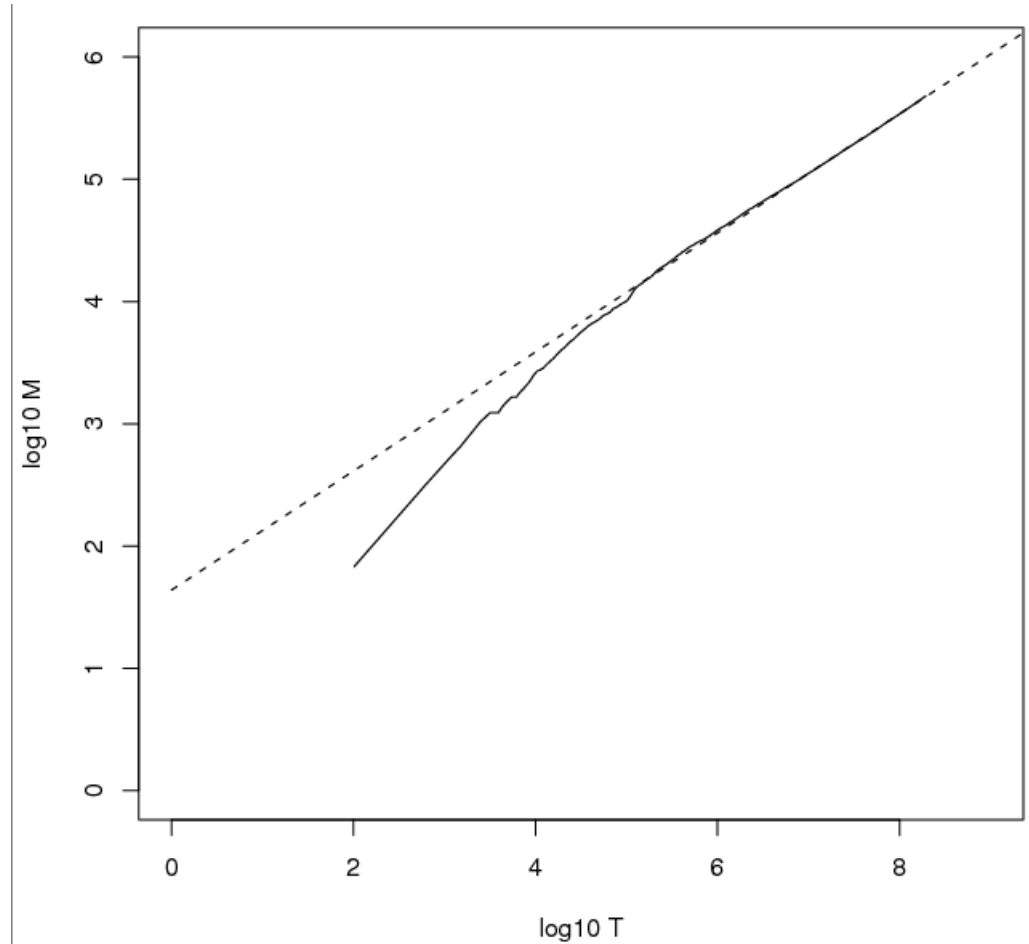


Estimating the number of terms

- **Heaps' law:** $M = kT^b$
- Estimates vocabulary size as a function of the collection size
- M is the size of the vocabulary; T is the number of tokens in the collection,
- Typical values: $30 \leq k \leq 100$ and $b \approx 0.5$
 - It is the simplest possible relationship between the two in log-log space
 - An empirical finding (“empirical law”)

Heaps' Law

- For RCV1, the dashed line $\log_{10} M = 0.49 \log_{10} T + 1.64$ is the best least squares fit.
- Thus, $M = 10^{1.64} T^{0.49}$ so $k = 10^{1.64} \approx 44$ and $b = 0.49$.
- Good empirical fit for Reuters RCV1 !
- For first 1,000,020 tokens, law predicts 38,323 terms; actually, 38,365 terms



Estimating the number of terms

Heaps' law suggests that

- (i) the dictionary size continues to increase with more documents in the collection, rather than a maximum vocabulary size being reached
- (ii) the size of the dictionary is quite large for large collections.



Zipf's law: Modeling the distribution of terms

- Heaps' law gives the vocabulary size in collections
- We also study the relative frequencies of terms
- In natural language, there are a few very frequent terms and very many very rare terms
- Zipf's law: The i th most frequent term has frequency proportional to $1/i$

$$cf_i \propto 1/i$$

- cf_i is collection frequency: the number of occurrences of the term t_i in the collection.

Zipf consequences

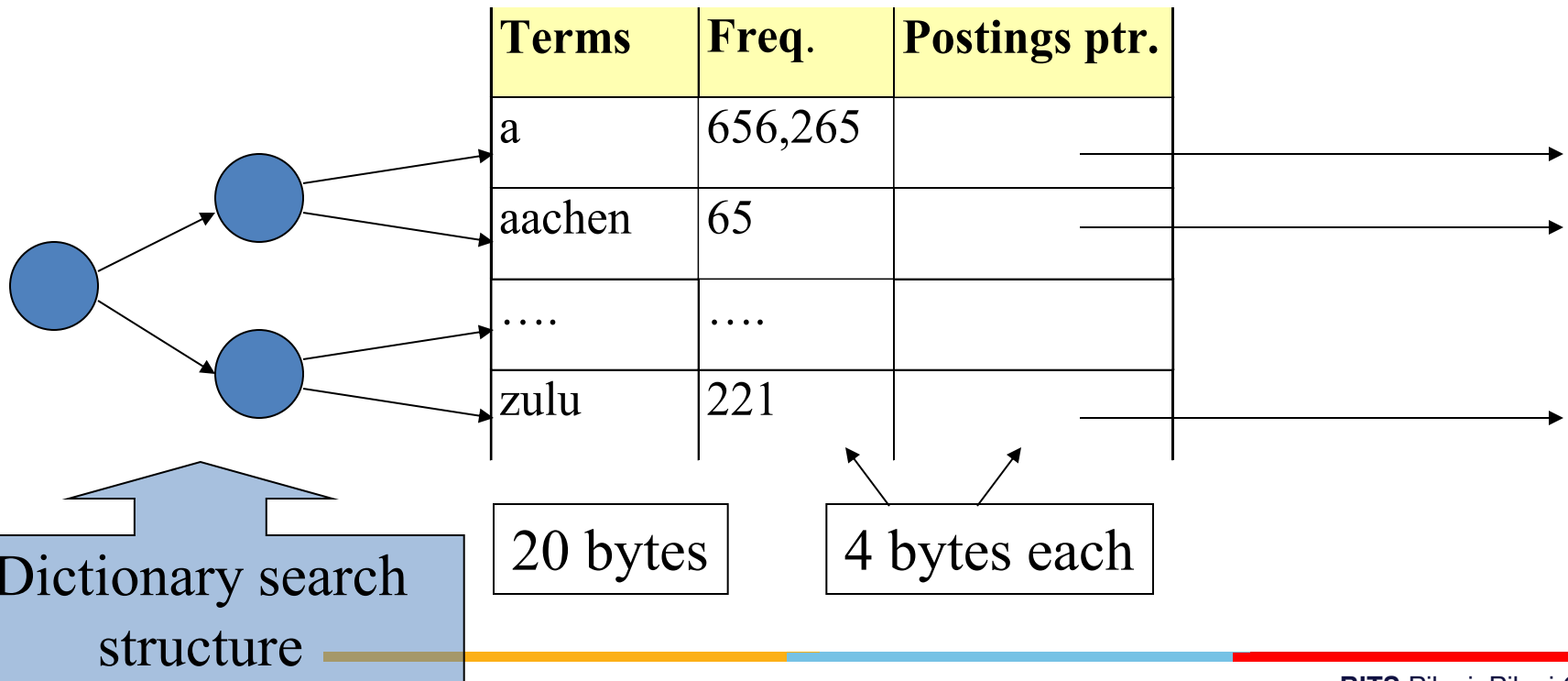
- If the most frequent term (*the*) occurs cf_1 times
 - then the second most frequent term (*of*) occurs $cf_1/2$ times
 - the third most frequent term (*and*) occurs $cf_1/3$ times ...
- Equivalent: $cf_i = K/i$ where K is a normalizing factor, so
 - $\log cf_i = \log K - \log i$
 - Linear relationship between $\log cf_i$ and $\log i$
- Another power law relationship

Dictionary Compression

Dictionary storage

Array of fixed-width entries

- ~400,000 terms; 28 bytes/term = 11.2 MB.



Dictionary storage

- Most of the bytes in the **Term** column are wasted – we allot 20 bytes for 1 letter terms.
 - And we still can't handle *supercalifragilisticexpialidocious* or *hydrochlorofluorocarbons*.
- Written English averages ~4.5 characters/word.
- Ave. dictionary word in English: ~8 characters
 - How do we use ~8 characters per dictionary term?
- Short words dominate token counts but not type average.

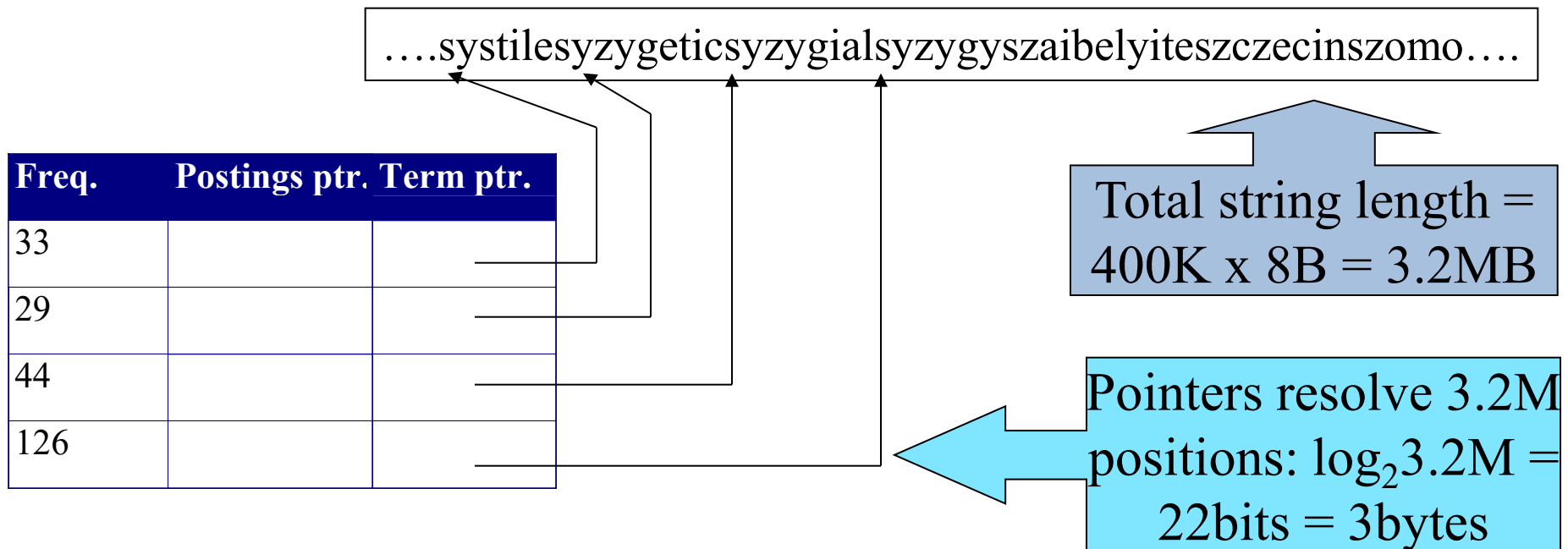
Compressing the term list: Dictionary-as-a-String

innovate

achieve

lead

- **Store dictionary as a (long) string of characters:**
 - Pointer to next word shows end of current word
 - Hope to save up to 60% of dictionary space.





Space for dictionary as a string

- 4 bytes per term for Freq.
 - 4 bytes per term for pointer to Postings.
 - 3 bytes per term pointer
 - Avg. 8 bytes per term in term string
 - 400K terms x 19 $\Rightarrow$ 7.6 MB (against 11.2MB for fixed width)
- } Now avg. 11 bytes/term, not 20.

Blocking

- Store pointers to every k th term string.
 - Example below: $k=4$.
- Need to store term lengths (1 extra byte)

....**7***systile***9***syzygetic***8***syzygial***6***syzygy***11***szaibelyite***8***szczecin***9***szomo*....

Freq.	Postings ptr.	Term ptr.
33		
29		
44		
126		
7		

} Save 9 bytes
on 3
pointers.

← Lose 4 bytes on
term lengths.

Net

- Example for block size $k = 4$
- Where we used 3 bytes/pointer without blocking
 - $3 \times 4 = 12$ bytes,
- now we use $3 + 4 = 7$ bytes.
- For $k = 4$, we save $(k - 1) \times 3 = 9$ bytes for term pointers, but need an additional $k = 4$ bytes for term lengths.
- The total space requirements for the dictionary of Reuters RCV1 are reduced by 5 bytes per four-term block, or a total of $400,000 \times 1/4 \times 5 = 0.5\text{MB}$, bringing us down to 7.1 MB.

Saved another $\sim 0.5\text{MB}$. This reduces the size of the dictionary from 7.6 MB to 7.1 MB.

We can save more with larger k .

Exercise

Estimate the space usage (and savings compared to 7.6 MB) with blocking, for block sizes of $k = 8$ and 16 .

Exercise

- **Estimate the space usage (and savings compared to 7.6 MB) with blocking, for block sizes of $k = 8$ and 16 .**

Solution:

- $K = 8$

We will save $:(8-1) * 3 = 21$ bytes for term pointer

Need additional $k=8$ for term length so space reduced by 13 bytes per 8 term block

Total space reduced by $= 400000 * 13 / 8 = 0.65$ MB

Total space : $7.6 - 0.65 = 6.95$ MB

- $K=16$

We will save $:(16-1) * 3 = 45$ bytes for term pointer

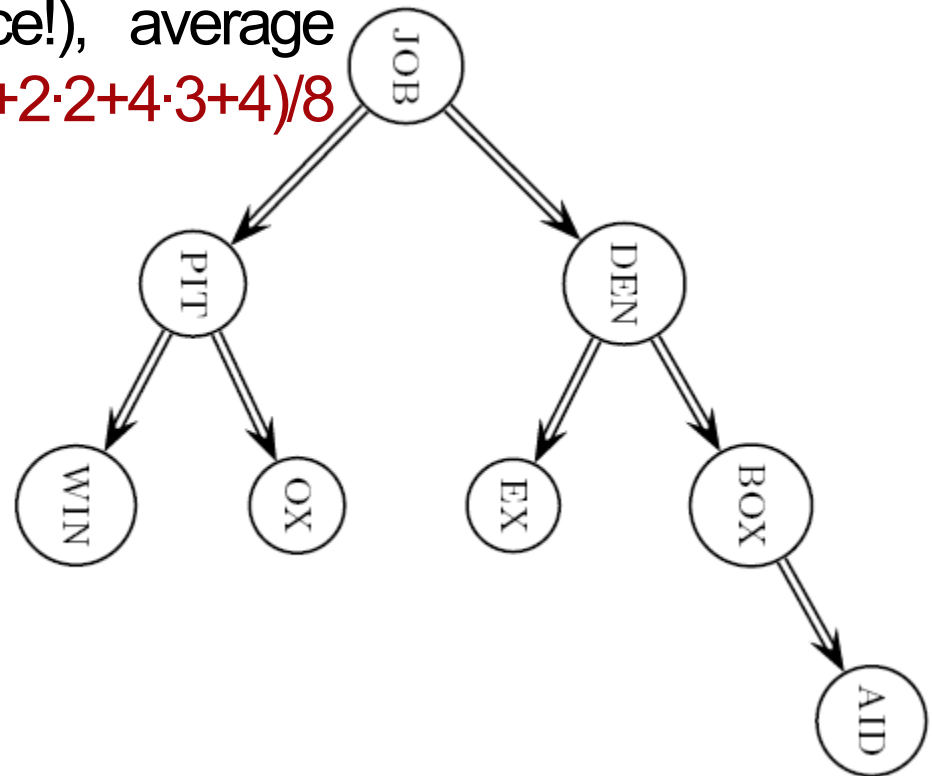
Need additional $k=16$ for term length so space reduced by 29 bytes per 16 term block

Total space reduced by $= 400000 * 29 / 16 = 0.725$ MB

Total space : $7.6 - 0.725 = 6.875$ MB

Dictionary search without blocking

Assuming each dictionary term equally likely in query (not really so in practice!), average number of comparisons = $(1+2\cdot 2+4\cdot 3+4)/8$
~2.6

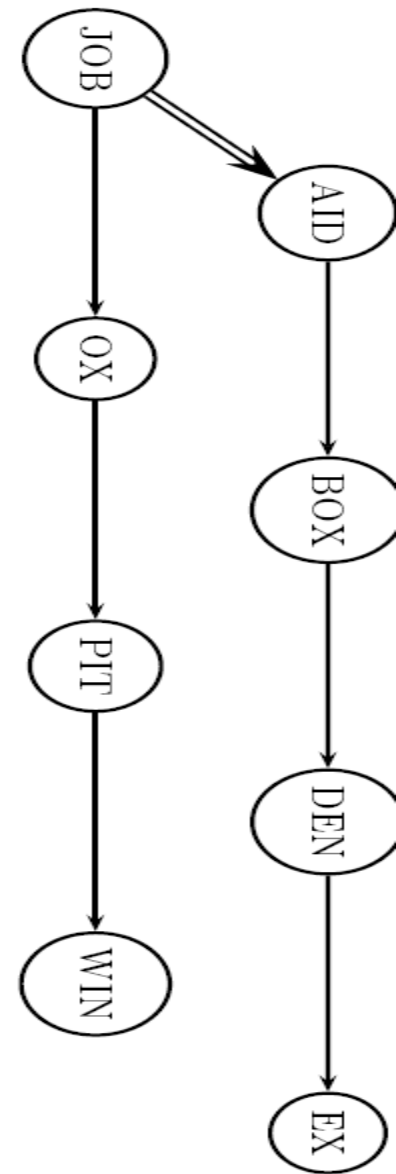


Dictionary search with blocking

Binary search down to 4-term block;

- Then linear search through terms in block.

Blocks of 4 (binary tree), avg. = $(1+2+2+2+3+2+4+5)/8 = 3$ compares



Front coding

Front-coding:

- Sorted words commonly have long common prefix – store differences only
- (for last $k-1$ in a block of k)

8*automata***8***automate***9***automatic***10***automation*

→ **8***automat****a****1**◇**e****2**◇**ic****3**◇**ion**

Encodes *automat*

Subsequent terms

Begins to resemble general string compression.

RCV1 dictionary compression summary

Technique	Size in MB
Fixed width	11.2
Dictionary-as-String with pointers to every term	7.6
Also, blocking $k = 4$	7.1
Also, Blocking + front coding	5.9



Postings compression

- The postings file is much larger than the dictionary, factor of at least 10
- **Key Point** : store each posting compactly
- A posting for our purposes is a docID
- For Reuters (800,000 documents), we would use 32 bits per docID, when using 4-byte integers
- Alternatively, we can use $\log_2 800,000 \approx 20$ bits per docID
- Our goal: use far fewer than 20 bits per docID.

Postings: two conflicting forces

- A term like ***arachnocentric*** occurs in maybe one doc out of a million – we would like to store this posting using $\log_2 1M \sim 20$ bits.
- A term like ***the*** occurs in virtually every doc, so 20 bits/posting is too expensive.
 - Prefer 0/1 bitmap vector in this case

Postings file entry

- We store the list of docs containing a term in increasing order of docID.
 - **computer**: 33,47,154,159,202 ...
- Consequence: it suffices to store *gaps*.
 - 33,14,107,5,43 ...
- Hope: most gaps can be encoded/stored with far fewer than 20 bits.

Three postings entries

	encoding	postings list				
THE	docIDs	...	283042	283043	283044	283045 ...
	gaps		1	1	1	...
COMPUTER	docIDs	...	283047	283154	283159	283202 ...
	gaps		107	5	43	...
ARACHNOCENTRIC	docIDs	252000	500100			
	gaps	252000	248100			

Variable length encoding

- Aim:
 - For ***arachnocratic***, we will use ~ 20 bits/gap entry.
 - For ***the***, we will use ~ 1 bit/gap entry.
- If the average gap for a term is G , we want to use $\sim \log_2 G$ bits/gap entry.
- Key challenge: encode every integer (gap) with about as few bits as needed for that integer.
- This requires a *variable length encoding*
- Variable length codes achieve this by using short codes for small numbers

Variable Byte (VB) codes

- For a gap value G , we want to use close to the fewest bytes needed to hold $\log_2 G$ bits
- Begin with one byte to store G and dedicate 1 bit in it to be a continuation bit c
- If $G \leq 127$, binary-encode it in the 7 available bits and set $c = 1$
- Else encode G 's lower-order 7 bits and then use additional bytes to encode the higher order bits using the same algorithm
- At the end set the continuation bit of the last byte to 1 ($c = 1$) – and for the other bytes $c = 0$.

Example

docIDs	824	829	215406
gaps		5	214577
VB code	00000110 10111000	10000101	00001101 00001100 10110001

Postings stored as the byte concatenation

000001101011100010000101000011010000110010110001

Key property: VB-encoded postings are uniquely prefix-decodable.

For a small gap (5), VB uses a whole byte.

Exercise



- What is the posting list that can be decoded from the variable byte-code
10001001 00000011 10000010 11111110 ?
- What is the largest gap that can be encoded in 2 bytes using the variable-byte encoding ?

Exercise



- What is the posting list that can be decoded from the variable byte-code

10001001 00000011 10000010 11111110 ?

9

386

126

RCV1 compression

Data structure	Size in MB
dictionary, fixed-width	11.2
dictionary, term pointers into string	7.6
with blocking, $k = 4$	7.1
with blocking & front coding	5.9
collection (text, xml markup etc)	3,600.0
collection (text)	960.0
Term-doc incidence matrix	40,000.0
postings, uncompressed (32-bit words)	400.0
postings, uncompressed (20 bits)	250.0
postings, variable byte encoded	116.0
postings, γ -encoded	101.0

EC-2 Sample Question Paper

Course No. : AIMLCZG537/DSECLZG537
Course Title : Information Retrieval

Q.1. (a) What is Term Frequency? Does Boolean retrieval system require Term Frequency? Justify your answer. [2]

(b) Compute the minimum edit distance using Levenshtein algorithm, between the terms **HEAP** and **WEEP**. Fill in the table given below by distances between all prefixes as computed by the algorithm. . [3]

		W	E	E	P
H					
E					
A					
P					

(c) Assume an IR application is designed to answer questions based on the document corpus. Which is better option to include in the preprocessing of the documents, Stemming or Lemmatization? Justify with an example. [2]

(d) Assume a biword index. Give an example of a document which will be returned for a query of “**artificial intelligence applications**” but is a false positive which should not be returned. [only relevant answers will be awarded mark]. [1]

(e) What is the Soundex code for the following two terms, **Square** and **Smoke**? Assume that the alphabets are mapped to numbers as follows: $(B, F, P, V \rightarrow 1)$, $(C, G, J, K, Q, S, X, Z \rightarrow 2)$, $(D, T \rightarrow 3)$, $(L \rightarrow 4)$, $(M, N \rightarrow 5)$ and $(R \rightarrow 6)$. **Show the computations clearly** (Mere answers will not fetch marks). [2]

[2+3+2+1+2=10]

Q.2. (a) What is Heap’s Law? . How is it useful for Inverted Index compression in Information Retrieval? [2]

(b) Given below is a snapshot of dictionary terms; Show how Front coding compression is applied on this with a block size of 4, with proper explanation. [3]

Battalion ;Batten; Batter; Battered; Battery; Batting; Battle; Battleship; Batty

(c) What happens when the block size is increased in dictionary compression? Discuss with an example. [2]

(d) Give the **Variable Byte encoding** for the following postings list of the index term;

random : 100, 109, 130, 345

[3]

[2+3+2+3=10]

Q.3. (a) What are the issues in using Jaccard coefficient for scoring? [2]

(b) Assume a collection statistic of N (Total No. of Documents) = 200,000, T (number of Tokens) = 500,000. For $n = 12$ splits, $r = 5$ segments, $j = 3$ partitions, in a distributed Index creation using MapReduce architecture,

i) What is the minimum number of parser machines needed ?

ii) What is the size of split allotted to the parser? [2]

(c) Consider the following documents: [6]

For each document the terms along with their term frequencies are given.

Doc1: Data (40), analysis (10), algorithms (20)

Doc2: Data (55), mining (100), techniques (80)

Doc3: Data (100), warehousing (45), algorithms (45)

Rank the above documents for the query **“Data mining Data warehousing”** using Cosine Similarity between the query and documents. Assume logarithmic term weighting for query and documents, no **idf** weighting for both query and documents and apply cosine normalization for both query and documents.

[2+2+6=10]
